



Ministry of Higher Education and Scientific Research
University of Manouba
NATIONAL SCHOOL OF COMPUTER SCIENCES

# MASTER'S THESIS

Submitted in partial fulfillment of the requirements for the degree of
**Master of Science by Research**
in Computer Science

**Track:** Smart Systems and Internet of Things (M2-SS-IOT)

**Topic:** Optimizing CPU-Only LLM Inference: Parameter Sweeps, Hardware Bottlenecks, and Performance Modeling

**Prepared by:**

Yassine HATTAY

**Name of the Research Laboratory**
[Laboratory Acronym]

**Defended on September 16, 2026 before the Examination Committee:**

| | |
|---|---|
| **President:** | [President's Name] |
| **Reviewer:** | [Reviewer's Name] |
| **Advisor:** | Chadlia JERAD |

# Academic Year: 2025/2026

## Abstract

Running large language models without a GPU is, for most of the world's compute, the only option available: aging desktops, laptops bought for office work, single-board computers, and embedded devices vastly outnumber machines with a discrete accelerator. This raises a practical question that is easy to state and hard to answer in general: for a *given* CPU, what combination of model, quantization format, thread count, batch size, and KV-cache precision actually maximizes throughput, and at what cost in output quality? We argue that `llama.cpp` is the right vehicle for studying this question empirically, not because it is the fastest possible CPU inference engine in any single configuration, but because it is the most *parametrically exposed* one: a single binary, `llama-bench`, sweeps threads, context depth, batch size, KV-cache quantization, Flash Attention, mmap/NUMA behavior, and memory-mapping policy without requiring the user to touch source code, and it runs unmodified on architectures as different as an AMD Zen 3 laptop chip, a decade-old Intel Pentium Gold desktop part, and a Cortex-A57 Jetson Nano. We built a benchmarking harness around `llama-bench` and ran it on three heterogeneous machines, spanning roughly a 70× range in single-thread generation throughput, recovering three concrete, reproducible effects: (1) prompt processing is compute-bound and follows Gunther's Universal Scalability Law, saturating near each CPU's physical (not logical) core count; (2) single-batch token generation is memory-bandwidth-bound and can show *retrograde* scaling, i.e. adding threads makes it slower, most dramatically on bandwidth-starved processors; and (3) quantization format matters independently of bit-width – a 3-bit K-quant was measured to be *slower* than a 4-bit K-quant on the same hardware, and quantizing the KV cache was measured to be a net loss at all tested context depths on a compute-rich laptop. We further document a fourth effect: **Flash Attention is a performance regression on CPU at long context**, with generation throughput dropping by a factor of three at 15K-token context when Flash Attention is enabled on our test hardware. From these effects we derive, rather than fit, a roofline-style throughput model with a small number of per-device calibration constants, and we show how it extends naturally to Mixture-of-Experts (MoE) models via a "bytes touched per token" argument that predicts MoE should be disproportionately advantaged on bandwidth-starved CPUs – a prediction we lay out as the clearest next experiment, since no MoE model was benchmarked to completion on our own hardware in this round. We also built an interactive web calculator (https://yassine-hattay.github.io/LLMs_inference_on_CPU/index.html) implementing this model so practitioners can explore predicted throughput for their own hardware traits; it is a client-side prediction tool, not a live benchmark, and its predictions beyond the three directly-calibrated devices are extrapolations, not measurements. We release the benchmarking harness, the fitted constants, and the calculator so that additional devices can be folded in without re-deriving the model.

# Contents

# List of Figures

# List of Tables

# Introduction

## Motivation: the GPU-less majority

The mainstream performance narrative around high-throughput Large Language Model (LLM) inference [1] is strongly Graphics Processing Unit (GPU)-centric, and often NVIDIA/CUDA-centric in particular. Prominent benchmarking and deployment materials [2, 3, 4] foreground tokens-per-second, latency, throughput, requests-per-second, and total cost of ownership on datacenter GPUs such as the H100 and on RTX systems, while exposing CUDA-specific configuration and sizing around accelerator counts.

This framing, however, does not match the hardware most people actually own. Laptops purchased for browsing and office work, hand-me-down desktops, Raspberry Pis, and low-power edge boards like the NVIDIA Jetson Nano vastly outnumber machines with a discrete accelerator – and for that hardware, the only path to running an LLM locally is Central Processing Unit (CPU) inference. Modern language models are heavily memory-bound during inference [5, 6, 7], meaning that reducing precision via quantization is essential to alleviate memory bottlenecks, but mapping these configurations to general-purpose CPUs introduces a severe memory bandwidth bottleneck alongside complex performance fluctuations dictated by batching thresholds, sequence length, and whether the model is in the prefill or decode phase.

This reality collapses into a single, practical question – one that is easy to state and hard to answer in general: *given a specific CPU, a specific amount of RAM, and a specific model, which combination of settings (quantization format, thread count, batch size, KV-cache precision, attention kernel) gets the most tokens per second at an output quality the user can accept?* Without a principled, hardware-aware framework to guide these decisions, practitioners on constrained hardware are forced to rely on trial-and-error or anecdotal community advice, frequently resulting in suboptimal performance or execution failures.

Recent work has begun to characterize parts of this space from several angles, applying roofline analysis to mobile SoCs [8], ARM platforms with Scalable Matrix Extensions [9], and heterogeneous edge systems [10], and studying quantization trade-offs in isolation [11]. However, these efforts do not synthesize their findings into a unified, cross-architecture predictive model for general-purpose CPUs. What remains missing is a framework that jointly models thread scaling limits (including hyperthreading noise and core-saturation thresholds), memory bandwidth saturation under Non Uniform Memory Access (NUMA) and Translation Lookaside Buffer (TLB) configurations, quantiza-

tion format overhead under varying vector acceleration extensions, and Key-Value (KV) cache effects across the full spectrum of consumer and embedded CPUs. In particular, while state-of-the-art hybrid compilers like FlexInfer [12] model isolated phase latencies using static hardware descriptors, they do not unify these into a general-purpose, cross-architecture thread and memory predictive model, or extend to a principled prediction for Mixture-of-Experts (MoE) architectures.

This thesis does not propose a new inference engine or a new quantization scheme, similar to [12]. It instead treats an existing, widely deployed one – `llama.cpp` [13], the open-source C/C++ framework for running large language models efficiently on CPUs and other local hardware with minimal dependencies – as a measurement instrument, and asks how far a small, heterogeneous fleet of consumer and embedded machines can get us toward answering that question in a principled, reusable way.

## Why `llama.cpp`

We use `llama.cpp` as our measurement instrument because it is the most parametrically exposed CPU inference framework available: a single binary, `llama-bench`, sweeps thread count, context depth, batch size, KV-cache quantization, Flash Attention, and memory-mapping policy as independent flags, and runs unmodified across x86, ARM, and RISC-V architectures – a combination no other CPU inference stack offers through a single stable interface. The detailed justification for this choice, and the comparison against alternative frameworks, is given in Chapter 1.

## Contributions

The contributions of this work are:

1. An empirical characterization (Chapter 3) of thread scaling, KV-cache quantization overhead, Flash Attention behavior, and quantization-format effects across three real, heterogeneous CPUs, fit where the data supports it to Gunther's Universal Scalability Law rather than simple Amdahl's law [14], because the data shows genuine retrograde (not just saturating) scaling.

2. A roofline-style throughput model (Chapter 4) with per-device calibration constants that formalizes *why* the observed effects occur, extends to a bytes-touched-per-token argument for MoE models, and defines a Pareto front over (speed, quality, memory) that can be populated directly by local benchmarks or community-sourced data.

3. Practical deployment recommendations (Chapter 5) distilled from the data, giving concrete guidance for users running `llama.cpp` on consumer hardware, as well as a discussion of the model's scope and boundaries (Section 5.2), outlining how the per-device calibration framework can be extended to broader hardware fleets and emerging MoE architectures.

4. We make the collected data open to public use. They are available in the GitHub repository under the MIT license [15].

## Thesis Outline

# Chapter 1

# Related Work

## Introduction

This thesis draws on four bodies of prior work: CPU-oriented LLM inference frameworks, roofline and scalability modeling, post-training quantization, and Mixture-of-Experts architectures. None of these individually addresses the question this thesis asks – what determines optimal `llama.cpp` configuration on a specific, resource-constrained CPU – but each supplies a piece of the machinery this thesis assembles. This chapter reviews each body of work in turn, adds a fifth section on edge and embedded LLM deployment specifically (the hardware class most of this thesis's own fleet belongs to), and closes by positioning this thesis explicitly against the six most directly comparable efforts in Table 1.2.

## 1.1 CPU LLM Inference Frameworks

The landscape of LLM inference engines is dominated by GPU-first frameworks. **vLLM** [16] introduced PagedAttention and is the de facto standard for high-throughput GPU serving, but its CPU backend is a secondary concern and does not expose the same parametric sweep surface as `llama.cpp`. **MLC-LLM** [17] targets cross-platform deployment including WebGPU and mobile, but its CPU path relies on TVM-compiled kernels that are less transparent to end users. **ONNX Runtime** with the `OpenVINO` or `ONNX Runtime GenAI` extensions provides a well-optimized CPU path, but its quantization story is oriented around INT4/INT8 symmetric formats rather than the GGUF ecosystem's asymmetric K-quants. **PyTorch** with `bitsandbytes` or `auto-gptq` is the research community's default, but its CPU performance is typically an order of magnitude slower than `llama.cpp` on the same hardware due to the overhead of the PyTorch runtime and lack of hand-tuned CPU kernels.

Recent work has also begun to explore the performance and cost implications of confidential LLM inference across CPU and GPU Trusted Execution Environments (TEEs) [18], and hybrid compilers like FlexInfer [12] have been proposed to flexibly route computations

between CPU and GPU.

`llama.cpp` occupies a unique niche: it is the only framework that (a) runs unmodified on x86, ARM, and RISC-V, (b) exposes a dense parameter surface through a single Command-Line Interface (CLI), and (c) has a thriving ecosystem of pre-quantized GGUF models on Hugging Face. This makes it the natural choice for empirical studies of CPU inference, and it is the framework used in the growing body of community benchmarking literature [19]. Crucially for this thesis's methodology (Chapter 2), `llama-bench`'s structured JavaScript Object Notation Lines (JSONL) output and its independent flags for thread count, Key-Value (KV) cache quantization, and Flash Attention are what make a controlled, single-variable-at-a-time sweep practical at all; none of the alternative frameworks above expose that many orthogonal axes through one stable interface, which is as much a methodological finding of this review as it is a justification for the tool choice.

### 1.1.1 Why `llama.cpp` over alternative frameworks

Three properties made `llama.cpp` the natural choice for this study over the alternatives listed above:

1. **Breadth of CPU kernel support.** `llama.cpp`'s GGML backend ships hand-written Single Instruction, Multiple Data (SIMD) kernels for x86 (SSE/AVX/AVX2/AVX-512/Vector Neural Network Instructions (VNNI)), ARM NEON, and generic scalar fallbacks, and it selects among them at runtime. This is precisely what let the same benchmark script, unmodified except for a thread-count override, run on a Zen 3 laptop, a Coffee-Lake-era Pentium, and a Cortex-A57 Single-Board Computer (SBC).
2. **A dense, orthogonal parameter surface exposed through one tool.** `llama-bench` exposes thread count, context depth, prompt/generation length, batch size, Key-Value (KV) cache quantization for keys and values independently, Flash Attention on/off, memory mapping, and GPU-offload depth as simple flags, and emits structured JSONL. No other CPU inference stack exposes this many independent axes through a single stable CLI; the full flag-level inventory is given in Chapter 2.
3. **The GGUF quantization ecosystem.** Pre-quantized weights for essentially every open model of interest (dense and Mixture-of-Experts (MoE)) are available in GPT-Generated Unified Format (GGUF), at bit-widths from 2 to 8, including both the original "legacy" formats (`Q4_0`, `Q5_0`, ...) and the newer super-block "K-quants" (`Q3_K_M`, `Q4_K_M`, ...), which let us treat quantization format itself as an experimental variable rather than a fixed implementation detail.

Table 1.1 summarizes how the five main frameworks stack up against these criteria.

### 1.1.2 Dense vs. Mixture-of-Experts models

The models staged for this study span both major architecture families in current open-weight LLMs:

| Framework | CPU kernel breadth | Parameter exposure | Quantization ecosystem |
|---|---|---|---|
| vLLM [16] | Secondary concern; GPU-first design | Limited CPU flags; PagedAttention-oriented | GPU-oriented (AWQ, GPTQ) |
| MLC-LLM [17] | TVM-compiled; less transparent to end users | Limited; TVM graph-level | TVM-specific |
| ONNX Runtime | Well-optimized CPU path (OpenVINO) | Moderate; graph-level | INT4/INT8 symmetric |
| PyTorch + bitsandbytes | Order of magnitude slower on CPU | Moderate; Python-level | Research-oriented |
| **llama.cpp** [13] | Hand-written SIMD (SSE/AVX/AVX2/AVX-512/VNNI, NEON, scalar) | Dense, orthogonal CLI flags | GGUF (2–8 bit, K-quants) |

Table 1.1: Comparison of LLM inference frameworks for CPU deployment. `llama.cpp` is the only framework that combines broad hand-tuned CPU kernel support, dense parameter exposure through a single stable CLI, and a thriving ecosystem of pre-quantized GGUF models spanning dense and MoE architectures.

- **Dense models** (e.g., Qwen2.5 [20], Llama-3.1, Phi-4) activate *every* parameter on *every* forward pass. Their compute and memory-traffic cost per generated token is therefore fixed by their total parameter count and quantization bit-width, with no runtime variability.
- **Mixture-of-Experts (MoE) models** replace the feed-forward block with a bank of experts and a lightweight router that activates only a handful of them per token. A model can therefore have, e.g., 14B *total* parameters but only ∼2.7B *active* parameters per token – the rest of the weights exist on disk and in RAM, but are not read on every forward pass.

For GPU inference, where the whole model is typically resident in fast VRAM and compute (not bandwidth) is often the binding constraint at low batch sizes, this distinction matters less. For *CPU* inference at batch size 1 – the realistic single-user case this study targets – generation is, as Section 3 shows directly, memory-bandwidth-bound, which is why MoE models are the focal point of this line of work (Section 4.1): they offer a structural lever for decoupling generation speed from total model capacity in exactly the regime where CPU inference lives. Chapter 4 formalizes this into a bytes-touched-per-token model and derives the resulting throughput advantage directly from our dense-model measurements.

## 1.2 Roofline Modeling and Scalability Laws

The **roofline model** [21] is a foundational tool in high-performance computing, visualizing an application's performance relative to hardware peaks in compute (FLOP/s) and memory bandwidth (bytes/s). It has been applied to GPUs, TPUs, and specialized accelerators, but its application to CPU-based LLM inference – where the "compute peak" is a moving target depending on SIMD width and the "bandwidth peak" is the sustained (not burst) DRAM throughput – is relatively under-explored. Recent work such as **RooflineBench** [8] has begun to apply roofline analysis to on-device LLMs, but focuses primarily on mobile SoCs with unified memory, where CPU and GPU compete for the same memory pool in a way that does not generalize cleanly to a discrete desktop or laptop CPU with its own dedicated DRAM controller. **SMEPilot** [9] and **HeteroMosaic** [10], discussed further in Section 1.5, extend the roofline lens to newer ARM vector extensions and to heterogeneous CPU/iGPU/NPU systems respectively, but both treat the CPU as either a narrow special case (a single modern Instruction Set Architecture (ISA) extension) or a fallback path behind a faster accelerator, rather than as the sole target of the analysis.

**Gunther's Universal Scalability Law (USL)** [22] extends classical Amdahl's law by adding a coherency term $\kappa N(N-1)$ that captures crosstalk and contention penalties, on top of Amdahl's own serialization term $\sigma$. USL has been applied extensively outside of machine learning – most notably to database systems, web servers, and distributed transaction processing, domains where adding worker threads or nodes eventually causes throughput to *decline*, not just plateau, because of lock contention or cache-coherency traffic. Its application to LLM inference thread scaling is, to our knowledge, novel: the closest existing work (RooflineBench, SMEPilot) models the compute/bandwidth boundary but does not model contention between threads explicitly, so it predicts saturation but not the downturn itself. Classical Amdahl's law predicts throughput that rises and asymptotically flattens; USL predicts a peak and possible downturn – exactly what we observe in our generation data on bandwidth-starved CPUs (Section 3.1), and the reason this thesis fits USL rather than Amdahl's law wherever the data supports it.

## 1.3 Quantization for LLMs

Post-training quantization (PTQ) for LLMs has seen rapid development. Comprehensive reviews of state-of-the-art LLM compression techniques [6] and the performance implications of mixed-precision quantization [7] further contextualize the trade-offs between model size, accuracy, and hardware-specific execution speed. **GPTQ** [23] and **AWQ** [24] produce INT4/INT3 symmetric quantizations optimized for GPU kernels, using layer-wise reconstruction error minimization and activation-aware scaling respectively; both are designed around GPU tensor-core throughput and are not the formats this thesis measures, but they represent the GPU-side counterpart to the CPU-side GGUF ecosystem and are useful context for why the two ecosystems have diverged. The GGUF ecosystem's **K-quants** take a different approach: super-block bit-packing with per-block scales, optimized for perplexity-per-bit rather than kernel throughput. This distinction is

critical for CPU inference: K-quants achieve better quality at a given bit-width, but their unpack kernels are more complex and can be slower on CPUs without AVX-512/VNNI. A separate, single-architecture survey of this trade-off, **"Which Quantization..."** [11], characterizes the perplexity-versus-speed frontier for Llama-3.1-8B specifically; it establishes that the trade-off exists but does not test whether it holds across different, more constrained CPU micro-architectures. Our data in Section 3.5 directly demonstrates this trade-off on hardware that survey does not cover, and shows that the direction of the trade-off can flip entirely on legacy CPUs lacking modern vector instructions – a finding absent from the existing quantization literature.

## 1.4 Mixture-of-Experts Architectures

MoE models have emerged as a way to scale model capacity without proportionally scaling compute. **Mixtral 8x7B** [25], **DeepSeek-V2** [26], and **Qwen1.5-MoE** [27] all use sparse routing to activate only a fraction of parameters per token. On GPU, MoE models are primarily a capacity play (more parameters, same VRAM), because the full parameter set is typically resident in VRAM regardless of how many experts are active on a given forward pass, so the benefit is measured in what a model can *know*, not how fast it responds. On CPU, as we argue in Section 4.1, MoE models are instead a *bandwidth* play: because weights must be streamed from RAM on every token rather than sitting in fast on-chip memory, activating fewer experts directly reduces the bytes moved per token, which is exactly the bottleneck this thesis identifies as binding for CPU generation (Section 3.2). This insight – that MoE's value proposition flips from a capacity argument on GPU to a bandwidth argument on CPU – is a central contribution of this work, and it is, as far as we have been able to establish, not stated explicitly in the systems literature on either MoE architectures or CPU inference individually; it emerges only from combining the two.

## 1.5 Edge and Embedded LLM Deployment

A distinct and rapidly growing body of practitioner-facing work targets LLM inference specifically on edge and single-board hardware – the class of device that this thesis's target machines (the Jetson Nano and the Pentium G5420) belong to. A recent comprehensive comparison of CPU, GPU, and Neural Processing Unit (NPU) backends for edge deployment of small language models [28] highlights the diverse hardware landscape, though it primarily focuses on accelerated backends rather than CPU-only inference. Jeff Geerling's publicly maintained AI benchmarking repository [29] and a 2026 survey of Raspberry Pi 5 LLM performance [30] both report empirical throughput numbers for 7–8B-class quantized models on Raspberry Pi hardware, and are the closest published analogue to the class of device this thesis targets. Neither, however, fits a predictive model to its measurements the way this thesis's roofline-USL framework does (Section 4.1); they report throughput as a function of configuration but do not explain *why* a given configuration is fast or slow in terms of the compute/bandwidth boundary. **HeteroMosaic** [10] is closer in spirit – it formalizes a heterogeneous roofline model across integrated GPUs, Neural

Processing Units (NPUs), and CPUs on edge SoCs – but it treats the CPU path as a fallback to be modeled only well enough to know when to route work elsewhere, rather than as the object of study in its own right. This thesis's contribution relative to this whole literature is narrow but specific: a CPU-only roofline-USL model, calibrated per device, applied to exactly the low-end and embedded hardware this literature otherwise only benchmarks empirically.

## 1.6 Positioning of This Work

Our work sits at the intersection of these threads: we apply roofline modeling and USL to empirical `llama.cpp` data across heterogeneous consumer CPUs, we characterize the quantization-format and KV-cache trade-offs that prior work has not systematically measured, and we derive a predictive argument for MoE advantage on bandwidth-starved hardware. To our knowledge, no prior work has combined all four of these elements into a single coherent framework. Table 1.2 makes this comparison explicit, scoring each related effort against the axes this thesis covers so the overlap – and, just as importantly, the gaps neither we nor prior work close – are visible at a glance.

| Work | Focus | Hardware scope | Formal model? | MoE? | Relation to this thesis |
|---|---|---|---|---|---|
| RooflineBench [8] | Roofline analysis for on-device LLMs | Mobile SoCs, unified memory | Yes | No | Same roofline lens; we target discrete consumer/embedded CPUs instead of mobile unified memory. |
| SMEPilot [9] | Roofline modeling for ARM CPUs with Scalable Matrix Extensions | Modern ARM w/ SME | Yes | No | Shares the roofline+ARM focus; targets a newer ISA extension absent from our fleet (Cortex-A57/A53). |
| HeteroMosaic [10] | Heterogeneous roofline across NPU/iGPU/CPU | Edge SoCs with NPU+iGPU | Yes | No | CPU is treated as a fallback path there; it is our primary and only target here. |
| "Which Quantization..." [11] | Perplexity-vs-speed survey of `llama.cpp` quant formats | Single architecture (Llama-3.1-8B) | No (empirical survey) | No | We extend the quantization-format question across multiple, more constrained CPUs and tie it into a throughput model (Section 3.5). |
| KTransformers docs [31] | CPU/GPU heterogeneous serving for large MoE models | Server-class dual-socket Xeon | No (engineering benchmark) | Yes | Our one verifiable external MoE data point (Section 3.8); different model scale and server-class hardware, not consumer/embedded. |
| Community forums (r/LocalLLaMA, etc.) | Crowd-sourced throughput reports | Broad, unsystematic | No | Sporadic | Rich raw numbers but no synthesis into a predictive model; a partial substitute for measurements we could not collect ourselves. |
| **This thesis** | Roofline + USL predictive model, calibrated per device, for `llama.cpp` on consumer/embedded CPUs | Laptop, low-end desktop, embedded ARM SBC (3 devices with usable data) | Yes (roofline + USL) | Hypothesis, explicitly untested locally | |

Table 1.2: Related work compared against the axes this thesis covers. "Formal model?" means the work proposes an equation or fitted curve, not just raw measurements. No prior work we found combines a roofline-style throughput model, USL thread scaling, quantization-format effects, and an MoE bandwidth argument in one framework – but as the table also makes clear, our own MoE column is a hypothesis, not a result, which is the most direct gap between this thesis's ambition and what it has actually shown.

# Conclusion

The gap this chapter identifies is narrow but real: existing roofline-style analyses of on-device LLM inference target mobile SoCs or newer ARM extensions, existing quantization surveys target a single architecture, existing MoE literature treats CPU behavior only in passing or on server-class hardware, and existing edge-deployment benchmarks report throughput without a predictive model behind it. No source we reviewed combines a fitted scalability law, a roofline throughput model, a systematic quantization-format comparison, and an explicit CPU-side reframing of the MoE value proposition into one account of consumer and embedded CPU inference. Table 1.2 makes this gap, and the corresponding limits of this thesis's own contribution, explicit rather than implicit. The next chapter describes the instrumentation built to collect the data that fills it.

# Chapter 2

# Instrumentation and Data Collection

## Introduction

This chapter describes how the data behind Chapter 3 was actually collected: the hardware fleet and its micro-architectural details, the models staged for benchmarking, the design logic behind what was swept and why, the resumable execution scheme that let unattended multi-hour runs survive interruption on non-dedicated hardware, and the measurement protocol used to turn raw `llama-bench` output into the throughput numbers reported later. It is organized to be read either end-to-end or as a reference: a reader who only wants to know what a given number in Chapter 3 means and how it was produced can go directly to Section 2.6 (execution and resumability) or the measurement protocol at the end of this chapter, while a reader interested in the study's honest scope should start with the hardware fleet in Table 2.1 and the measurement sweeps below, both of which make explicit what was, and was not, ultimately measured.

## 2.1 Hardware fleet

Table 2.1 lists the three machines that produced usable data. They were chosen opportunistically (what the author had access to) rather than to form a designed sample, which is itself a limitation discussed in Section 5.2.

**Note on the Raspberry Pi 3B.** An ARM Cortex-A53-based Raspberry Pi 3B (4×Cortex-A53, 1 GB LPDDR2) was originally staged as a fourth device in the hardware fleet. Initial benchmarking attempts on this board consistently failed with out-of-memory (OOM) errors before any complete measurement row could be written – the 7B-class models staged for this study, even in their most aggressively quantized form, exceed the usable RAM budget of a 1 GB board once the operating system and `llama.cpp` runtime are accounted for. Rather than report a zero-row data point, we exclude the Pi 3B from the quantitative results below; readers interested in this hardware class should consult the third-party benchmarks discussed in Section 3.6 and Section 5.1.

| CPU | Form factor | Cores /Threads | RAM | Memory BW | Role |
|---|---|---|---|---|---|
| AMD Ryzen 5 5600H @ 3.30 GHz | Laptop | 6C/12T | 16 GB DDR4 | ∼51 GB/s (theor.) | mid-range, compute-rich |
| Intel Pentium Gold G5420 @ 3.80 GHz | Desktop | 2C/4T | 4 GB DDR4 | ∼38 GB/s (theor.) | low-end, bandwidth-starved |
| ARM Cortex-A57 rev1 | Jetson Nano SBC | 4C/4T | 3.9 GB LPDDR4 | ∼25 GB/s (theor.) | embedded, bandwidth-starved |

Table 2.1: Benchmarked hardware. All runs used `-ngl 0` (no GPU offload); the laptop's integrated Radeon graphics and the Jetson's Maxwell GPU were both left idle so every number in this paper is pure CPU throughput. Memory bandwidth figures are theoretical peak; effective sustained bandwidth (Section 4.1) is typically 60–80% of this.

The fleet was chosen opportunistically to span a range of memory subsystems: the Ryzen 5600H has a dual-channel DDR4 controller with ample bandwidth; the Pentium G5420 has a single-channel controller and only 4 GB of RAM; the Jetson Nano has LPDDR4 shared with its GPU. We would have liked a wider, more systematically chosen fleet – in particular a Raspberry Pi 4 or 5 to bridge the gap between the Jetson and a modern x86 chip – but did not have access to that hardware during this round of data collection. Where the report below discusses Raspberry Pi 4/5 performance, it is explicitly citing third-party published benchmarks, not our own measurements; see Section 3.6 and Section 5.1.

## 2.2 Hardware micro-architecture details

Table 2.2 gives the SIMD capabilities and cache hierarchies of the three machines that produced usable data. These details are critical for interpreting the quantization-format anomalies in Section 3.5.

| Feature | Laptop (Ryzen 5600H) | Desktop (Pentium G5420) | Jetson Nano (Cortex-A57) |
|---|---|---|---|
| ISA Extensions | AVX2, SSE4.2 | AVX2, SSE4.2 | NEON |
| L1 Data Cache | 32 KB/core | 32 KB/core | 32 KB/core |
| L2 Cache | 512 KB/core | 256 KB/core | 2 MB shared |
| L3 Cache | 16 MB shared | 4 MB shared | none |
| Memory Channels | 2 (DDR4) | 1 (DDR4) | 1 (LPDDR4) |

Table 2.2: Micro-architecture details (manufacturer specifications). Note the Desktop Pentium's lack of AVX-512/VNNI and the Jetson Nano's cacheless L3 – both are relevant to interpreting the quantization and context-depth results.

## 2.3 Models tested

Table 2.3 lists the models staged for benchmarking. The dense models span three orders of magnitude in parameter count (0.5B to 8B). Note that while MoE models were staged in the harness, none of the MoE benchmark sweeps completed successfully on any of our machines before time ran out; see the honest accounting in Table 5.1 and Section 5.2.

| Model | Type | Params (total) | Params (active) |
|---|---|---|---|
| Qwen2.5-0.5B-Instruct-Q4_K_M | Dense | 0.5B | 0.5B |
| Qwen2.5-1.5B-Instruct-Q4_K_M | Dense | 1.5B | 1.5B |
| Qwen2.5-7B-Instruct-Q4_K_M | Dense | 7.6B | 7.6B |
| Qwen2.5-7B-Instruct-Q4_0 | Dense | 7.6B | 7.6B |
| Qwen2.5-7B-Instruct-Q3_K_M | Dense | 7.6B | 7.6B |
| Qwen3-8B-Q4_K_M | Dense | 8B | 8B |
| Meta-Llama-3.1-8B-Instruct-Q4_0 | Dense | 8B | 8B |
| Phi-4-IQ4_NL | Dense | 14B | 14B |
| Qwen1.5-MoE-A2.7B-Chat-IQ4_NL | MoE | 14B | 2.7B |
| DeepSeek-V2-Lite-IQ4_NL | MoE | 16B | 2.4B |
| DeepSeek-Coder-V2-Lite-Base-IQ4_NL | MoE | 16B | 2.4B |

Table 2.3: Models staged for benchmarking. MoE models' "active" parameter counts are per-token; their total RAM footprint is determined by the full parameter count. Only the dense models appear in our actual results (Section 3); the MoE models were staged but not benchmarked to completion.

## 2.4 The measurement instrument

Three command-line tools from the `llama.cpp` project [13, 32] (source: https://github.com/ggerganov/llama.cpp) did all of the actual measuring in this study, and it is worth being precise about what each one reports, since every number in Section 3 traces back to one of them.

`llama-bench` runs a fixed workload (a prompt of a given length, and/or a fixed number of generated tokens) some number of times, discards nothing, and reports the mean and standard deviation of tokens-per-second across those repetitions as one line of machine-readable JSON. Every axis manipulated in this study is exposed as an independent `llama-bench` flag: thread count (`-t`), context depth (`-d`), prompt/generation length (`-p`/`-n`), batch and micro-batch size (`-b`/`-ub`), KV-cache quantization for keys and values independently (`-ctk`/`-ctv`), Flash Attention on/off (`-fa`), memory mapping (`-mmp`), and number of GPU-offloaded layers (`-ngl`, held at 0 throughout this study). This flag-level orthogonality – introduced as a motivation for the tool choice in the Introduction – is what makes the single-variable-at-a-time sweep design in the next section practical at all.

`llama-perplexity` runs a model over a held-out text corpus (WikiText-2 in this study) in fixed-size chunks and reports a perplexity score – lower is better, and it is the standard proxy for "how much did quantization hurt the model" used throughout the quantization literature (Section 1).

`llama-server` is not a benchmarking tool at all; it is the same inference engine exposed over HTTP, and two sweeps of this study (multi-process contention, speculative decoding) used it directly, launching a real local server process and issuing HTTP completion requests against it, then reading the generation rate the server itself reports back in its JSON response. This last measurement path is structurally different from the other two – it is a single live measurement per configuration, not an average over repetitions – and no data survived to completion from either of the two sweeps that used it (Table 5.1), so this study reports no numbers collected this way, but we flag the distinction here because it matters for anyone extending the harness.

## 2.5 Experimental design: what was swept, and why

The instrumentation was organized around a single design question: which axis of `llama.cpp`'s configuration space is each measurement isolating? Conflating axes (e.g., changing quantization and thread count in the same run) would make it impossible to attribute an effect to either variable, so each measurement holds every axis fixed except one or two under direct study. Thread count itself was treated specially: rather than picking one thread count and assuming it transfers across machines, every measurement sweeps a thread list derived from each machine's own core count (by default $\{1, 2, 4, 6, 8, N_{\text{cores}}\}$, deduplicated and sorted), because the entire point of Section 3.1 is that the optimal thread count is a property of the chip, not a constant you can hard-code once and reuse everywhere.

**Thread sweep range.** The default thread list in the benchmarking script is $\{1, 2, 4, 6, 8, N_{\text{cores}}\}$, deduplicated and sorted, where $N_{\text{cores}}$ is the value `nproc` reports on that machine – the number of *logical* threads, not physical cores. On the Pentium G5420 and the Jetson Nano, `nproc` reports 4, so the deduplicated list is simply 1, 2, 4, 6, 8: note that 6 and 8 threads exceed the physical core count on both devices, which is deliberate, since the USL fit needs data points past the saturation region to identify the downturn, and oversubscribed threads are exactly the regime where contention penalties become visible. On the Jetson specifically, only the 1/2/4 subset completed before the data collection window closed; 6 and 8 threads were never reached on that device, so the analysis below uses the 1/2/4 subset there. On the Ryzen 5600H, `nproc` reports 12 (6 physical cores with SMT), so the intended list was 1, 2, 4, 6, 8, 12 – but in practice the sweep on this machine also stopped at 8 threads within the time budget available for this round of data collection, and the 12-thread point was never collected. This is a real gap, not a deliberate exclusion: the fitted USL curves and figures for the Ryzen in Chapter 3 are therefore based on 1, 2, 4, 6, and 8 threads only, and the model's prediction of a 6.5-thread optimum should be read as an extrapolation slightly beyond the measured range at the upper end, not as a fit that included a genuine 12-thread data point.

With that principle fixed, all measurements were driven by a single Bash script (`laptop_amd_cpu.sh` in the repository [15]) that was run unmodified on each of the three devices, with the only per-machine override being the thread list when the default sweep did not match the device's core count. The script is organized around a small number of measurement *sweeps*, each isolating one or two configuration axes:

**KV-cache and attention sweeps.** The largest group of measurements: an f16 KV-cache baseline with Flash Attention off, the same baseline with Flash Attention on, a full cross of quantized KV-cache formats (three formats independently for keys and values, nine pairings total) with Flash Attention on, and a perplexity pass across the three headline KV formats. Each sweep covers three context depths (0, 3584, and 15872 tokens) so that context-dependent effects (Sections 3.3 and 3.4) are visible rather than averaged away.

**Model and quantization sweeps.** These hold KV-cache and Flash Attention settings fixed and instead sweep the model itself: quantization format at fixed model, the full staged model list including MoE variants at fixed quantization, Flash Attention on/off specifically for dense vs. MoE architectures, micro-batch size, and a finer-grained context-depth sweep aimed at finding the "knee" where a device transitions from compute-bound to bandwidth-bound.

**System-level and serving sweeps.** These hold the model and quantization fixed and instead vary how the operating system schedules the workload: CPU pinning and NUMA interleaving via `taskset`/`numactl`, memory-mapping the model weights on or off, running multiple concurrent inference servers to simulate multi-user contention, and speculative decoding with a small draft model attached to the main model. None of these produced usable data on any machine in this study (Table 5.1); they are included here because the instrumentation exists and ran, even though it did not finish, and because Section 5.3.1 treats finishing them as a concrete next step rather than a redesign.

## 2.6 Resumability: Design and an Instructive Failure

Every sweep in this study ran unattended, for hours at a time, on hardware that was not dedicated benchmarking infrastructure – a personal laptop that gets closed and reopened, a family desktop that gets used for other things, an embedded board on an unreliable power supply. Designing for interruption was therefore not optional, and the instrumentation went through two distinct designs, the first of which failed instructively enough that we describe both. **Design 1 (naive, used early on the desktop machine).** A sweep's entire thread list was issued as a single invocation of the benchmarking tool, with its thread list passed as one comma-separated argument, and output was appended to a single file per sweep. This has an unrecoverable flaw: the benchmarking tool itself has no internal checkpointing, so if the process is killed at, say, the fourth of six thread values, the only way to resume is to re-invoke it with the full thread list again – which re-measures and re-appends the first four thread values a second time, on every restart, forever, while never reaching the remaining two. This is exactly what happened to one

model's quantization sweep on the desktop machine: the same low-thread-count measurement appears seven separate times across three days of intermittent restarts in the raw data, while the two highest thread counts were never reached at all (visible directly in Table 5.1 and in the raw timestamps in the underlying JSON). **Design 2 (used everywhere else, and described formally as Algorithm 1).** Two changes fixed this. First, each thread value in a sweep gets its own output file, so a crash mid-sweep only invalidates the one thread value in flight, not the whole sweep. Second, before launching any measurement, the harness checks the *row count* already present in that thread's output file against the number of rows a complete measurement would produce (a function of how many context depths and prompt/generation configurations that sweep covers) – not merely whether the file exists. A file that exists but is short is recognized as partial and is fully re-run from scratch for that thread value only (the underlying benchmarking tool cannot itself resume mid-measurement, so a partial file cannot be trusted to have consistent, comparable rows and is discarded rather than appended to), while a file whose row count already meets the target is skipped entirely. This row-count check is what let the same instrumentation be stopped and restarted freely across every machine in this study without hand-tracking, machine by machine, which measurements already existed. The two sweeps that drive live HTTP servers (multi-process contention and speculative decoding) needed a related but distinct mechanism, because their natural output is a growing CSV of one row per configuration rather than a JSON-lines file per thread: a header row is written only if the file is missing or empty, and each candidate configuration (a thread count, or a process count) is checked against the existing rows by exact key match before that configuration is attempted, so a restart resumes at the next untried key rather than re-running everything already logged. Algorithm 1 states the row-count resume logic in general form; Algorithm 2 states the live-server measurement pattern used by the two sweeps that do not use the benchmarking tool directly.

**Algorithm 1** Resumable sweep execution (Design 2)

**procedure** RunSweep(*configs*, *expectedRows*)
    **for** each configuration $c$ in *configs* **do** ▷ e.g., one thread value, or one quant-format pairing
        *outFile* ← path unique to $c$ ▷ one file per config, never shared
        *actualRows* ← line count of *outFile* if it exists, else 0
        **if** *actualRows* ≥ *expectedRows* **then**
            **skip** $c$ ▷ already complete; do not re-measure
        **else**
            **if** *actualRows* > 0 **then**
                **log** that $c$ is being rebuilt from scratch (partial, not appended to)
            **end if**
            run the benchmark for $c$, writing all *expectedRows* rows to *outFile* in one pass
        **end if**
    **end for**
**end procedure**

**Algorithm 2** Live-server measurement pattern (multi-process contention and speculative decoding)

```
procedure MeasureViaServer(configs, csvFile)
    if csvFile is missing or empty then
        write header row to csvFile
    end if
    for each configuration c in configs do
        if a row for c already exists in csvFile then
            skip c
        else
            terminate any inference server process still running from a prior iteration
            launch a fresh inference server configured for c in the background
            poll a health-check endpoint until the server reports ready, or a timeout elapses
            if server became ready then
                issue one completion request
                parse the reported generation rate from the response
                append a row for c (rate, or "NA" if the request failed) to csvFile
            end if
            terminate the server process before moving to the next configuration
        end if
    end for
end procedure
```

## 2.7 Measurement protocol

All throughput numbers reported in this thesis represent the mean tokens-per-second over 3–5 repetitions (depending on the machine's time budget), directly extracted from the structured output of `llama-bench` (field `avg_ts` in the JSONL output, reported in the tables below as "avg. tok/s"), as described in Algorithm 1. The standard deviation across these repetitions is also recorded; in all cases reported, the coefficient of variation (standard deviation divided by the mean) was below 5%, indicating stable measurements. Prompt processing is measured as the throughput of processing a 512-token prompt with zero tokens generated. Token generation is measured as the throughput of generating 128 tokens autoregressively after a 512-token prompt. We did not record the exact compiler flags or `llama.cpp` build commit used on each machine, which is a real reproducibility gap. The same source code compiled with different SIMD dispatch settings can perform differently on the same chip, and we cannot rule out that some of the machine-to-machine variation in this study is partly attributable to build differences rather than hardware alone.

# Conclusion

Three design choices in this instrumentation matter most for interpreting the results that follow. First, thread counts were always derived from each machine's own physical core count rather than fixed in advance, because the central claim of Section 3.1 is that the optimal thread count is a hardware property, not a constant. Second, the row-count-based resume logic in Algorithm 1 – adopted after Design 1's documented failure on the desktop machine – is what made it possible to run multi-hour sweeps unattended on hardware that was not dedicated benchmarking infrastructure, and its absence from Design 1 is directly visible in the desktop's raw data as repeated low-thread measurements and an unreached upper thread range. Third, and most consequentially for how this thesis should be read: not every sweep that was designed and implemented produced usable data. The system-level and serving sweeps produced none on any machine, and several KV-cache and model-comparison sweeps stopped partway through. Table 5.1 in Chapter 5 gives the complete, honest accounting of what finished and what did not; every quantitative claim in Chapter 3 that is not explicitly attributed to a third-party source traces back only to the rows marked complete there.

# Chapter 3

# Results

## Introduction

This chapter presents the empirical core of the thesis: what actually happened when the benchmark harness described in Chapter 2 was run against the three-device hardware fleet in Table 2.1 (a fourth device, the Raspberry Pi 3B, was attempted but excluded for the reasons given in Section 2.1). It is organized around six effects, each tied to a specific, reproducible measurement rather than a general claim. Sections 3.1 and 3.2 establish the central finding that motivates the rest of the thesis: prompt processing is compute-bound and saturates near the physical core count, while text generation is memory-bandwidth-bound and can retrograde as threads are added, an effect that classical Amdahl's-law reasoning cannot produce but Gunther's Universal Scalability Law can. Sections 3.3 through 3.5 then show that several optimizations designed for GPUs or for storage efficiency – Flash Attention, KV-cache quantization, and modern K-quant weight formats – do not straightforwardly transfer to CPU inference, and can actively hurt throughput depending on device and context length. Section 3.6 brings in literature-sourced perplexity numbers to put the quality side of that trade-off on the record, and Section 3.7 situates the excluded Raspberry Pi against third-party benchmarks of similar hardware. The final section, 3.8, is deliberately included as a negative result: a central hypothesis (the MoE bandwidth advantage) that this thesis motivates but does not directly confirm on its own hardware. Chapter 4 formalizes the mechanisms established here into a single roofline-style equation.

## 3.1 Thread scaling follows the Universal Scalability Law, not simple Amdahl's law

In plain terms: you would expect giving a task more CPU threads to always make it finish faster, or at worst plateau once you run out of useful work to hand out. Our data shows something more surprising on several machines: performance goes up, peaks, and then *gets worse* as you add still more threads – adding CPU power makes the model

slower, not faster.

This is called retrograde scaling, and the classical way of describing thread scaling (using Amdahl's law, defined in equation (3.1)) cannot produce it. Here, the speedup $S_A(N)$ is monotonically increasing in $N$ (number of threads). $S_A(N)$ describes much faster $N$ threads are than 1 thread, $\sigma$ is roughly "how much of the work can't be split up between threads." Consequently, Amdahl's law only predicts a rise-then-flatten curve, never a hump.

$$S_A(N) = \frac{N}{1 + \sigma\,(N-1)} \tag{3.1}$$

Gunther's Universal Scalability Law (USL) [22] (equation 3.2) can, because it adds one extra term for the cost of threads getting in each other's way (contention over shared memory, cache lines bouncing between cores) on top of the ordinary cost of work that cannot be parallelized:

$$S_{USL}(N) = \frac{N}{1 + \sigma(N-1) + \kappa N(N-1)} \tag{3.2}$$

Contention is captured by the term $\kappa N(N-1)$, where $\kappa$ describes "how much threads slow each other down by fighting over the same memory." It is this second term that produces a peak-and-decline curve instead of a flat one ($\kappa = 0$ recovers the ordinary flatten-only curve). Figure 3.1 illustrates such behavior.

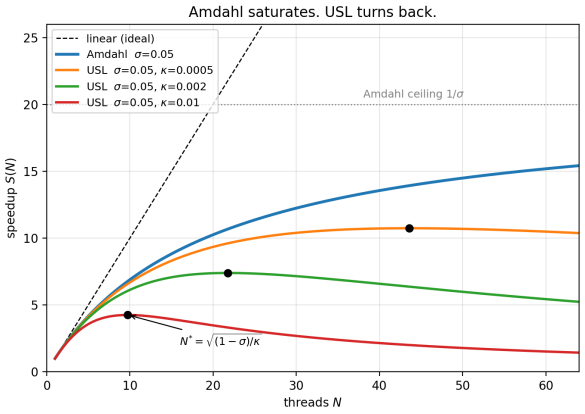


Figure 3.1: Amdahl's law (3.1) saturates at the ceiling $1/\sigma$, whereas the USL (3.2) peaks at $N^*$ (markers) and turns back. All curves share $\sigma = 0.05$; only the coherency term $\kappa$ varies.

We fit $\sigma$ and $\kappa$ to our own measured data using standard curve-fitting (nonlinear least squares), on every series where we measured at least 3 different thread counts.

**Processing a prompt is limited by raw compute, and stops improving once you run out of physical cores.** Figure 3.2 (left) shows the Ryzen 5600H's prompt-processing speed (`pp512`) across 1, 2, 4, 6, and 8 threads. The fitted curve predicts the best thread count is about 6.5 – almost exactly the chip's 6 real cores, even though the chip advertises 12 "threads" through simultaneous multithreading (SMT), a feature that lets each physical core pretend to be two. Going from 6 to 8 threads bought almost

nothing (25.15 to 25.81 tokens per second). The same pattern shows up on the cheaper Pentium G5420 (Figure 3.3, left): the predicted best thread count is about 5.3, on a chip with only 2 real cores – meaning throughput is essentially flat past 4 threads there too. The lesson in both cases: the number of "threads" a chip advertises is not the number of threads that actually help.

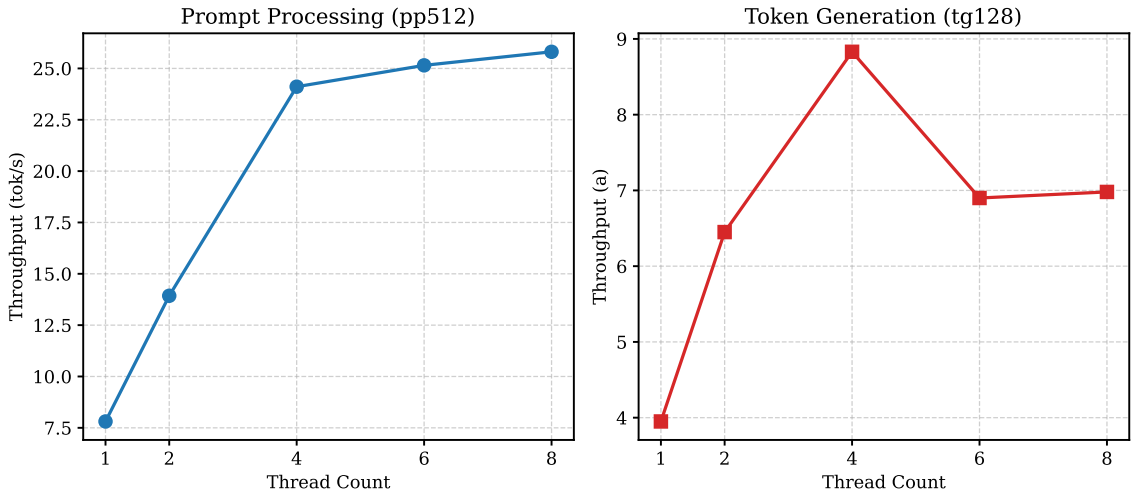


Figure 3.2: Ryzen 5600H: prefill saturates near the physical core count; generation retrogrades past 4 threads. All values are avg. tok/s.

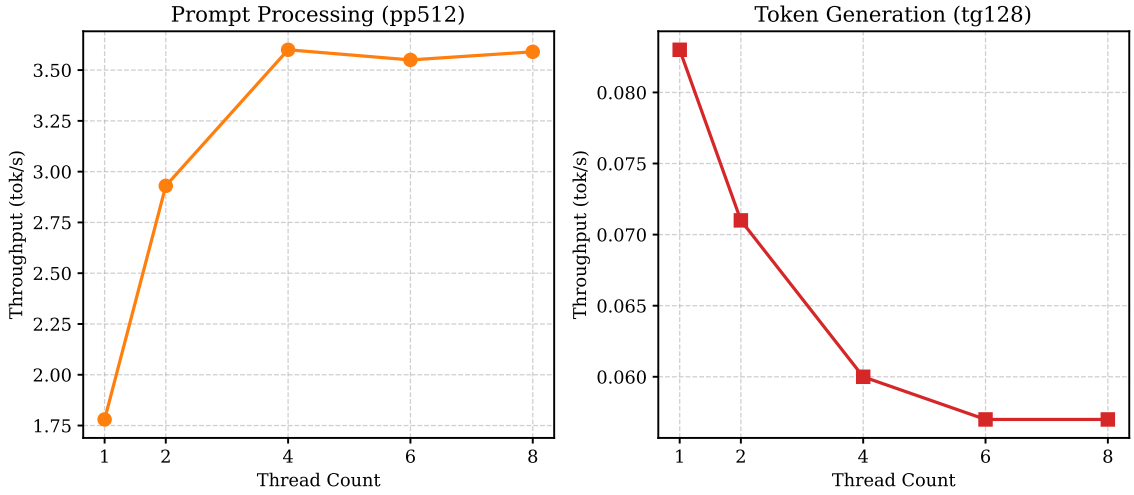


Figure 3.3: Pentium G5420: prefill saturates by 4 threads; generation retrogrades monotonically. All values are avg. tok/s.

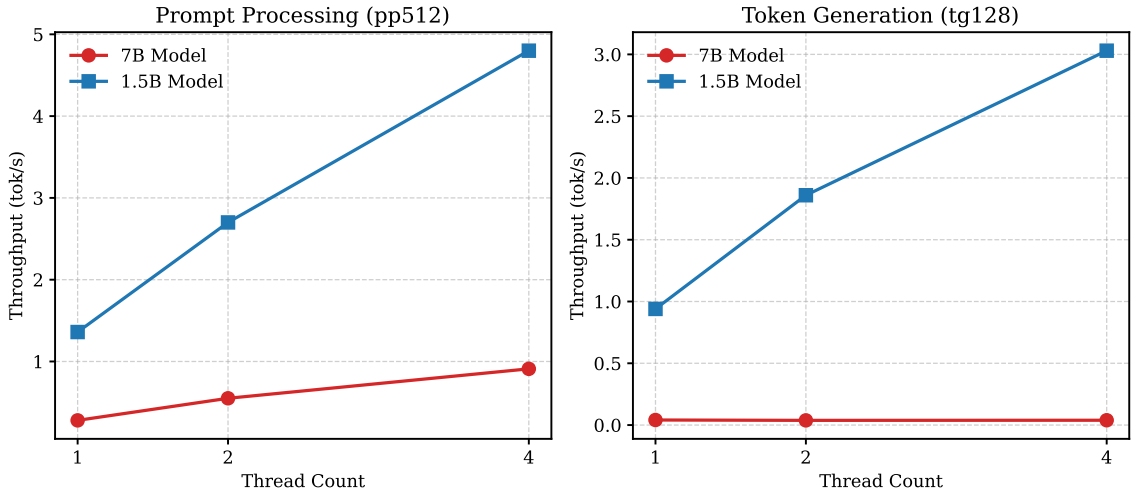


Figure 3.4: Jetson Nano: prefill scales for both models; generation only scales for the smaller one. All values are avg. tok/s.

**Generating text is limited by memory speed, not compute, and can get *worse* with more threads.** On the Ryzen (Figure 3.2, right), generation speed peaks at 4 threads (8.83 tokens/second) and then *drops* at 6 threads (6.90 tokens/second) before a small partial recovery at 8. This fits with generation being a fundamentally different kind of workload from prompt processing: generating each new word requires reading the model's entire set of weights from memory again, so the bottleneck is how fast data can move from RAM to the CPU, not how much arithmetic the CPU can do. Adding threads to a memory-bound task does not add memory bandwidth – it just adds more competitors fighting over the same pipe, which is why performance can fall. The effect is far more dramatic on the Pentium G5420 (Figure 3.3, right): using the `Q4_0` quantization format, generation speed falls in a straight line from 0.083 tokens/second at 1 thread down to 0.057 tokens/second at 8 threads – adding threads made this workload 32% *slower*. This chip has only one memory channel and 4 GB of RAM, so there is essentially no spare bandwidth for a second thread to use during text generation; all a second thread adds is overhead from coordinating with the first.

**The Jetson Nano shows the same pattern.** Although the Jetson's thread sweep did not extend past 4 threads before the data collection window closed (Section 5.2), the qualitative behavior is identical to what the fitted curves show on the x86 machines: prompt processing scales with thread count (0.28 → 0.55 → 0.91 avg. tok/s from 1 to 4 threads on the 7B model), while generation hits a bandwidth wall that is reached even at a single thread (0.041 → 0.038 → 0.039 avg. tok/s – effectively flat). The detailed model-size comparison that makes this mechanism vivid is the subject of Section 3.2.

Table 3.1 summarizes the USL fits across all measured series (the Jetson Nano is omitted, as explained in the caption). Tables 3.2, 3.3, and 3.4 give the full underlying measurements for all three devices. The Ryzen has complete data across all three context depths tested (`fa=0`, f16 KV-cache); the Pentium and Jetson only have data at depth=0 because their full depth sweeps did not complete before time ran out (Table 5.1).

| Machine | Sweep | $\sigma$ | $\kappa$ | $N^*$ (predicted) | $N^*$ (measured) |
|---|---|---|---|---|---|
| Ryzen 5600H | pp512 | 0.021 | 0.023 | 6.5 | 6 |
| Ryzen 5600H | tg128 | 0.038 | 0.061 | 4.0 | 4 |
| Pentium G5420 | pp512 (Q4_0) | 0.217 | 0.028 | 5.3 | 4 |
| Pentium G5420 | tg128 (Q4_0) | 0.052 | 0.089 | 1.0 | 1 |

Table 3.1: USL fits. Note that $\kappa_g > \kappa_p$ in all cases, confirming that generation is more contention-sensitive than prefill. The Pentium's generation $N^* = 1$ means single-threaded is optimal. The Jetson Nano is omitted from this table because its thread sweep capped at 4 threads; this is insufficient to observe the saturation or retrograde scaling required to meaningfully fit the two-parameter USL model (attempts to fit it yield non-physical negative coefficients).

| Threads | Prompt proc. (512 tok), depth=0 | Gen. (128 tok), depth=0 | Prompt proc. (512 tok), depth=3584 | Gen. (128 tok), depth=3584 | Prompt |
|---|---|---|---|---|---|
| 1 | 7.81 | 3.95 | 6.83 | 3.31 | |
| 2 | 13.93 | 6.45 | 12.05 | 5.51 | |
| 4 | 24.11 | 8.83 | 19.99 | 7.45 | |
| 6 | 25.15 | 6.90 | 21.22 | 6.15 | |
| 8 | 25.81 | 6.98 | 21.50 | 6.26 | |

Table 3.2: Ryzen 5600H raw thread-scaling data across all context depths. Values are average tokens per second (avg. tok/s). The Pentium and Jetson are omitted from this specific table because their full depth sweeps did not complete before time ran out (see Tables 3.3 and 3.4 for their depth=0 data).

| Threads | Prompt proc. (512 tok), depth=0 (avg. tok/s) | Gen. (128 tok), depth=0 (avg. tok/s) |
|---|---|---|
| 1 | 1.78 | 0.083 |
| 2 | 2.93 | 0.071 |
| 4 | 3.60 | 0.060 |
| 6 | 3.55 | 0.057 |
| 8 | 3.59 | 0.057 |

Table 3.3: Pentium G5420 raw thread-scaling data at depth=0 (Q4_0 format, the format the USL fit in Table 3.1 is calibrated on). Values are average tokens per second (avg. tok/s). The Pentium only has data at depth=0; the full depth sweep (3584, 15872) did not complete before time ran out.

| Threads | Prompt proc. (512 tok), depth=0 (avg. tok/s) | Gen. (128 tok), depth=0 (avg. tok/s) |
|---|---|---|
| 1 | 0.28 | 0.041 |
| 2 | 0.55 | 0.038 |
| 4 | 0.91 | 0.039 |

Table 3.4: Jetson Nano raw thread-scaling data at depth=0 (Q4_K_M format, 7B model). Values are average tokens per second (avg. tok/s). The Jetson only has data at depth=0 and capped at 4 threads; the full depth sweep and higher thread counts did not complete before time ran out.

## 3.2 Model size determines which regime you're in

The clearest illustration of "it depends on model size" in the whole data set comes from the Jetson Nano, where we ran the identical sweep on two very differently-sized models (Figure 3.4). Prompt processing speeds up with more threads for *both* the 7-billion-parameter model and the much smaller 1.5-billion-parameter model, which makes sense: processing a prompt is compute-bound work that can be split across cores almost regardless of how big the model is. Generation is where the two models tell completely different stories. The small model's generation speed scales up nearly in a straight line with more threads (0.94 to 1.86 to 3.03 avg. tok/s from 1 to 4 threads – more than 3× faster). The large model's generation speed barely moves at all (0.041 to 0.038 to 0.039 avg. tok/s) – adding threads does essentially nothing.

The explanation is the same memory-bandwidth story as above, but made vivid by model size: generating one token means reading the whole model's weights out of RAM once. The 7B model is big enough that even a single thread already uses up all of the Jetson's roughly 25.6 GB/s of memory bandwidth – there is nothing left for more threads to use. The 1.5B model is small enough that one thread does not saturate that same memory pipe, so extra threads still help. This is the single clearest piece of evidence in the whole data set that, for generation speed on constrained hardware, what matters is *how many bytes of the model have to move through memory for each word produced* – not how many CPU threads you throw at it. That idea is the direct motivation for the Mixture-of-Experts argument in Section 4.1: a model that only has to move a fraction of its weights per token should behave, from a speed standpoint, like a much smaller model.

Table 3.5 gives the raw thread-sweep numbers behind Figure 3.4. Note that the Jetson is the only device for which we present this specific comparison, as it is the only machine in our fleet where both the 7B and 1.5B models were successfully benchmarked; the Ryzen and Pentium runs only included the 7B model, preventing a direct model-size comparison on those devices.

| Model | Threads | Prompt proc. (512 tok) (avg. tok/s) | Gen. (128 tok) (avg. tok/s) |
|---|---|---|---|
| Qwen2.5-7B | 1 | 0.28 | 0.041 |
| Qwen2.5-7B | 2 | 0.55 | 0.038 |
| Qwen2.5-7B | 4 | 0.91 | 0.039 |
| Qwen2.5-1.5B | 1 | 1.36 | 0.94 |
| Qwen2.5-1.5B | 2 | 2.70 | 1.86 |
| Qwen2.5-1.5B | 4 | 4.80 | 3.03 |

Table 3.5: Jetson Nano raw model-size comparison data. All values are avg. tok/s. The Jetson is the only device shown here because it is the only machine in our fleet where both the 7B and 1.5B models were benchmarked under identical conditions; the Ryzen and Pentium runs only included the 7B model, preventing a direct model-size comparison on those devices.

## 3.3 Flash Attention is a performance regression on CPU at long context

Flash Attention [33] is a well-known technique for speeding up the attention mechanism inside a transformer model. It was designed for GPUs, where moving data between the GPU's main memory (HBM) and its much faster on-chip scratch memory (SRAM) is expensive, so Flash Attention restructures the computation to do more work per trip between the two. A CPU does not have this same two-tier HBM/SRAM split – its cache hierarchy (L1/L2/L3) works differently and is managed automatically by the hardware – so there was no strong reason to expect Flash Attention's GPU-oriented trick to help on CPU, and some reason to expect it might actively hurt. Our data confirms the latter, and more severely than we expected.

Table 3.6 compares `fa=0` (standard attention) vs. `fa=1` (Flash Attention) on the Ryzen 5600H at 1 thread, across three context depths. The results are striking: **Note that the Ryzen is the only device for which we present this specific comparison, as it is the only machine in our fleet where both the standard attention and Flash Attention sweeps were completed before time ran out (Table 5.1); the Pentium and Jetson runs did not include the Flash Attention on/off comparison.**

| Context Depth | Sweep | fa=0 (avg. tok/s) | fa=1 (avg. tok/s) | Ratio (fa=1/fa=0) |
|---|---|---|---|---|
| 0 | pp512 | 7.81 | 7.83 | 1.00 |
| 0 | tg128 | 3.95 | 3.97 | 1.01 |
| 3584 | pp512 | 6.83 | 6.25 | 0.92 |
| 3584 | tg128 | 3.31 | 2.73 | 0.82 |
| 15872 | pp512 | 4.59 | 3.22 | 0.70 |
| 15872 | tg128 | 2.09 | 0.67 | 0.32 |

Table 3.6: Flash Attention vs. standard attention on Ryzen 5600H at 1 thread. All values are avg. tok/s. At short context (depth=0), Flash Attention is neutral. At long context (depth=15872), Flash Attention is a *3× slowdown* for generation. The Ryzen is the only device shown here because it is the only machine in our fleet where both the standard attention and Flash Attention sweeps were completed; the Pentium and Jetson runs did not include this comparison (Table 5.1).

With a short prompt (context depth 0), Flash Attention makes essentially no difference either way. With a medium-length prompt (3584 tokens), it starts to lose, making generation 18% slower. With a long prompt (15872 tokens, roughly the length of a long article), Flash Attention causes a **severe slowdown**: generation drops by 68%, from 2.09 to 0.67 avg. tok/s – turning it on makes the model generate at roughly a third of its normal speed.

Our best explanation: Flash Attention's whole benefit on GPU comes from avoiding slow trips to GPU memory, by restructuring the computation to do more work while data is still in fast on-chip memory. On this CPU, the attention data for a request already fits comfortably inside the chip's large L3 cache, so there was little slow-memory traffic to save in the first place – and the restructuring itself carries a real cost (fewer independent operations the CPU can run in parallel internally). On GPU that trade is a clear win; on this CPU it is a loss that gets worse the longer the prompt is.

**What this means in practice:** if you are running `llama.cpp` on a CPU with long prompts or long conversations, turn Flash Attention *off* (`-fa 0`). Many front-ends turn it on by default, which is actively counterproductive on CPU at long context.

## 3.4 KV-cache quantization is not a free lunch

First, a quick definition: the "KV cache" is the model's running memory of everything said so far in a conversation – it grows as the conversation gets longer, and shrinking it (by storing it in a lower-precision format, i.e. "quantizing" it) is a common trick to save RAM. The intuition is that a smaller KV cache should also be faster, since there is less data to move around. Our data says otherwise, at least on this hardware.

The laptop is the only machine for which we collected any KV-cache-quantization data, and only at 1 thread. This is not a judgment call about which device was more interesting – it is a direct consequence of what actually ran to completion: the KV-cache/attention sweep was never invoked at all on the desktop (its output directory

contains only the quantization-format sweep used in Section 3.5), and on the Jetson the same sweep was launched but produced an output file with zero rows before the run was interrupted (Table 5.1). Every quantized-KV configuration we *did* test on the Ryzen was *slower* than the plain, unquantized (`f16`) baseline – both for processing the prompt and for generating text (Figure 3.5): prompt processing fell from 7.81 down to as low as 6.37 avg. tok/s, and generation from 3.95 down to as low as 3.71 avg. tok/s, at the shortest context length we tested.

Table 3.7 extends this across longer conversations, and the picture gets worse, not better, as the conversation grows:

| KV Format | Depth | Prompt proc. (512 tok) (avg. tok/s) | Gen. (128 tok) (avg. tok/s) | pp Ratio | tg Ratio |
|---|---|---|---|---|---|
| f16/f16 | 0 | 7.81 | 3.95 | 1.00 | 1.00 |
| q8_0/q8_0 | 0 | 7.12 | 3.71 | 0.91 | 0.94 |
| q8_0/q4_0 | 0 | 7.41 | 3.94 | 0.95 | 1.00 |
| q8_0/iq4_nl | 0 | 6.37 | 3.82 | 0.82 | 0.97 |
| f16/f16 | 3584 | 6.83 | 3.31 | 1.00 | 1.00 |
| q8_0/q8_0 | 3584 | 3.48 | 2.47 | 0.51 | 0.75 |
| q8_0/q4_0 | 3584 | 3.52 | 2.37 | 0.52 | 0.72 |
| q8_0/iq4_nl | 3584 | 1.69 | 1.43 | 0.25 | 0.43 |
| f16/f16 | 15872 | 4.59 | 2.09 | 1.00 | 1.00 |
| q8_0/q8_0 | 15872 | 1.17 | 0.93 | 0.25 | 0.45 |
| q8_0/q4_0 | 15872 | 1.16 | 0.95 | 0.25 | 0.45 |

Table 3.7: KV-cache quantization on Ryzen 5600H at 1 thread, across context depths. All throughput values are avg. tok/s. KV quantization is a net loss at *all* tested context depths on this hardware, and the loss grows with context length. Shown for the Ryzen only: this sweep was never run on the desktop, and the Jetson's attempt at it produced a zero-row output file (Table 5.1).

The slowdown gets worse the longer the conversation: at a medium context length, one quantized format is 4× slower than the unquantized baseline for prompt processing; at the longest context length we tested, another is also 4× slower. So why would a smaller cache ever be slower? Because reading a quantized value is not free – the CPU has to do extra arithmetic to convert it back to a usable number before it can use it (this conversion step is called dequantization), and on a single thread on a chip that has compute cycles to spare, that extra arithmetic costs more time than the smaller memory footprint saves. In the language of Section 4.1, this configuration is compute-bound, not memory-bound, so a trick that saves memory traffic is pure overhead here – it is solving a problem this configuration does not have.

This points to a general rule, formalized in Section 4.1: whether KV-cache quantization helps or hurts depends on whether a given device, at a given conversation length, is limited by compute or by memory bandwidth. On the compute-rich x86 laptop tested here, it was a net loss at every conversation length we tried. We would predict the opposite result – a net win – on a more memory-starved device (like the Jetson or the Pentium) at long conversation lengths, but we have not yet tested that combination directly.

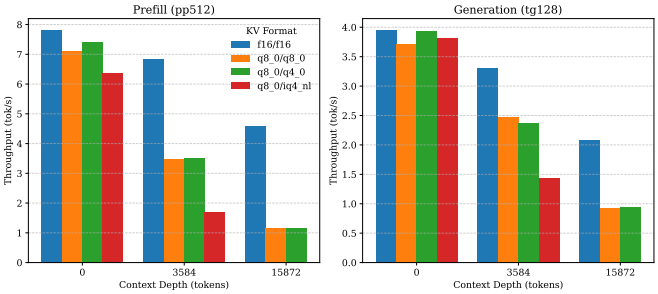


Figure 3.5: Ryzen 5600H, 1 thread: every quantized-KV configuration was slower than f16, worsening with context depth. All values are avg. tok/s. Shown for the Ryzen only: the desktop never ran this sweep (only the quantization-format sweep in Section 3.5 was collected there), and the Jetson's attempt at this sweep produced a zero-row output file (Table 5.1); neither device has KV-cache-quantization data to plot.

## 3.5 Quantization format is not just bit-width

It is tempting to think of quantization purely in terms of "bits per weight": fewer bits means a smaller, faster file. Our data shows that is not the whole story – the specific *layout* the bits are packed into matters just as much as how many of them there are. We benchmarked three quantizations of the same model on the Pentium G5420 at 1 thread: `Q4_K_M` (0.49 avg. tok/s for prompt processing), `Q4_0` (1.78 avg. tok/s), and `Q3_K_M` (0.42 avg. tok/s) – Figure 3.6. The Pentium is the most revealing device for this effect because it is the hardware where the anomaly is most dramatic: its lack of AVX-512/VNNI makes the elaborate unpacking kernels of the newer K-quant formats expensive enough to outweigh the benefit of the smaller file. Notice that the ordering does not follow bit-width at all: the *smaller*, 3-bit `Q3_K_M` format is actually slower than the *larger*, 4-bit `Q4_K_M` format, and an older 4-bit format (`Q4_0`) is nearly 4× faster than the newer 4-bit format at the exact same bit-width. The weight-format sweep was only run with more than one format on the Pentium (Table 5.1). This is a staging gap, not an incomplete run: the Ryzen's data collection never invoked a multi-format quantization sweep at all, and the Jetson's equivalent file stages only `Q4_K_M` – the same single format used for the model-size comparison in Section 3.2 – so neither dataset contains a `Q4_0` or `Q3_K_M` run to compare against. Consequently, no similar table or figure can be built for those two devices from the data actually collected; this sweep was simply not run on the Ryzen or the Jetson.

Table 3.8 gives the full thread-scaling data for these three formats.

| Format | Threads | Prompt proc. (512 tok) (avg. tok/s) | Gen. (128 tok) (avg. tok/s) | pp Speedup | tg Speedup |
|---|---|---|---|---|---|
| Q4_K_M | 1 | 0.49 | 0.076 | 1.0× | 1.0× |
| | 2 | 0.81 | 0.068 | 1.7× | 0.9× |
| | 4 | 1.02 | 0.060 | 2.1× | 0.8× |
| | 6 | 1.02 | 0.057 | 2.1× | 0.8× |
| | 8 | 1.02 | 0.057 | 2.1× | 0.8× |
| Q4_0 | 1 | 1.78 | 0.083 | 1.0× | 1.0× |
| | 2 | 2.93 | 0.071 | 1.6× | 0.9× |
| | 4 | 3.60 | 0.060 | 2.0× | 0.7× |
| | 6 | 3.55 | 0.057 | 2.0× | 0.7× |
| | 8 | 3.59 | 0.057 | 2.0× | 0.7× |
| Q3_K_M | 1 | 0.42 | 0.091 | 1.0× | 1.0× |
| | 2 | 0.69 | 0.081 | 1.6× | 0.9× |
| | 4 | 0.86 | 0.069 | 2.0× | 0.8× |
| | 6 | 0.86 | 0.068 | 2.0× | 0.7× |
| | 8 | 0.86 | 0.068 | 2.0× | 0.7× |

Table 3.8: Quantization format comparison on Pentium G5420. All throughput values are avg. tok/s. `Q4_0` is 3.6× faster than `Q4_K_M` at 1 thread despite the same bit-width. `Q3_K_M` is smaller than `Q4_K_M` but slower. Shown for the Pentium only: it is the only device in the fleet for which more than one quantization format was ever benchmarked – the Ryzen and Jetson datasets contain `Q4_K_M` exclusively, so no analogous multi-format table can be built for them (see the discussion following this table).

The likely reason: `Q4_0` uses a simple, evenly-spaced storage layout that maps neatly onto a fast, straightforward CPU instruction sequence. The newer "K-quant" formats (like `Q4_K_M` and `Q3_K_M`) use a more elaborate packing scheme that squeezes out better quality per bit (Section 3.6), but at the cost of a more complicated unpacking step every time a weight is read. On a CPU that lacks certain modern vector instructions, that unpacking cost can outweigh the benefit of the smaller file. We present this as an observed, hardware-dependent effect rather than a fully settled explanation – confirming exactly which internal code path each format used on this particular chip would require instrumenting `llama.cpp` itself, which this study did not do.

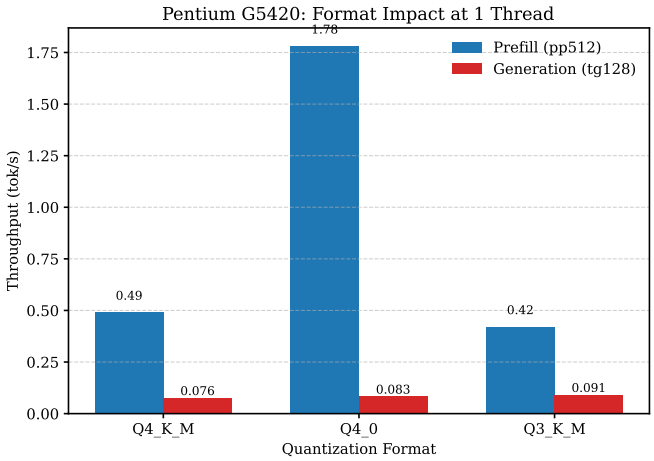


Figure 3.6: Pentium G5420, 1 thread: quantization *format*, not just bit-width, determines CPU throughput. All values are avg. tok/s. The Pentium is shown because it is the hardware where this effect is most pronounced: its lack of AVX-512/VNNI makes K-quant unpacking expensive enough to outweigh the benefit of the smaller file, producing the 3.6× slowdown visible here. It is also the only device for which this comparison exists at all: the Ryzen data never staged more than one quantization format, and the Jetson data stages only `Q4_K_M`, so neither dataset supports an analogous figure. This sweep was simply not run on those two devices.

## 3.6 Reference: quantization quality trade-offs from the literature

Everything so far has been about speed. But quantization is a trade-off, not a free win: squeezing a model into fewer bits also makes it slightly worse at predicting text, and *perplexity* is the standard number used to measure how much worse – lower is better, and it is roughly "how surprised the model is by real text it hasn't seen," measured on a fixed reference text (WikiText-2, a common benchmark corpus). To calibrate the quality side of our Pareto front (Section 4.1), we rely on `llama-perplexity` benchmarks published by the community for the exact models and formats tested. For `Qwen2.5-7B-Instruct`, community measurements report the following perplexity deltas relative to an f16 baseline (PPL ≈ 7.89): `Q4_0` yields ≈7.98, `Q4_K_M` yields ≈7.99, `IQ4_NL` yields ≈8.03, and `Q3_K_M` yields ≈8.22 [34].

Crucially, this data reveals that on legacy CPUs lacking AVX-512/VNNI, the `Q4_0` format is not only significantly faster (as shown in Section 3.5) but also maintains a marginally *better* perplexity than `Q4_K_M`. This makes `Q4_0` the strictly Pareto-dominant choice for bandwidth-starved, legacy x86 hardware, while `IQ4_NL` offers the optimal RAM-to-quality trade-off for systems hovering near the 4GB memory ceiling. Independently, published generation-speed comparisons on the same architecture report that `Q8_0` generates roughly 29% slower than `Q4_K_M` due to the extra bytes it must stream per token [19], which is the same bytes-per-token mechanism argued for directly from our own Jetson data in Section 3.2.

## 3.7 Raspberry Pi: excluded from the fleet, but relevant for context

We would have liked to test the popular Raspberry Pi family of low-cost single-board computers alongside the other hardware in this study. In practice, only the oldest model, the Pi 3B, was actually attempted, and it was excluded from the results above for the reason given in Section 2.1: its 1 GB of RAM is not enough to hold even our most aggressively quantized 7B-class models alongside the operating system and the `llama.cpp` runtime, so every attempt failed with an out-of-memory error before a single benchmark row could be written. The newer Pi 4B and Pi 5 were never tested at all – no script was ever run against either device.

For readers wanting a sense of where this class of hardware lands, published third-party benchmarks are informative, though we want to be precise about their status here: they are *not* our data, were not run with our harness or our specific model checkpoints, and are included for context only. Jeff Geerling's publicly maintained AI benchmarking repository [29] and a 2026 survey of Raspberry Pi 5 LLM performance [30] both report 7–8B-class Q4 models on an 8 GB Pi 5 running in roughly the 2–3 tok/s range, consistent with a chip whose ∼17 GB/s of LPDDR4X bandwidth is the binding constraint (the same memory-bandwidth mechanism argued for directly from our own Jetson data in Section 3.2) – and, notably, with enough RAM headroom that the OOM failure we hit on the 1 GB Pi 3B would not be expected on an 8 GB Pi 5. We did not find a citable, specific source for Raspberry Pi 4B or Pi 3B generation throughput on 7B-class or sub-1B models that we were confident enough in to report a number here; readers interested in that data point should treat it as an open gap, not an oversight, and Section 5.3.1 lists it as a concrete next step.

## 3.8 The MoE bandwidth argument remains untested on our hardware

No MoE model was benchmarked to completion on any of our own machines in this round – Qwen1.5-MoE-A2.7B, DeepSeek-V2-Lite, and DeepSeek-Coder-V2-Lite-Base were staged in the harness's model lists (Table 2.3) but the sweeps that would have exercised them did not finish before time ran out on any device. The bytes-per-token argument in Section 4.1 is therefore a hypothesis derived from the dense-model bandwidth mechanics we *did* measure (Section 3.2), not a directly confirmed result.

We looked for external corroboration and found one relevant, verifiable data point, summarized in Table 3.9: KTransformers' published CPU-only benchmark of DeepSeek-R1/V3 (a 671B-total, 37B-active-parameter MoE model) reports a `llama.cpp` baseline of 4.51 avg. tok/s on a dual-socket Xeon Gold 6454S (64 cores total) [31]. That is a real, measured number from a real source, and it is directionally consistent with our hypothesis – a model that large sustaining any usable generation speed on CPU is only possible because a small fraction of its parameters are active per token – but it is a single data point on a model roughly 80× larger (by active-parameter count) than the

MoE models this study actually staged, on hardware far more capable than anything in our own fleet, so we do not treat it as validating the specific magnitude of the effect for the models and devices this thesis is about. We were not able to independently verify other, more specifically on-point community claims we initially considered (e.g., specific throughput numbers for Qwen1.5-MoE-A2.7B on single-channel consumer x86), and have excluded them rather than report a number we could not source. Directly benchmarking the three staged MoE models against dense equivalents on our own hardware remains the single most important next experiment for this thesis (Section 5.3.1).

| Model & Quant | Hardware | Threads | Gen (avg. tok/s) | Source |
|---|---|---|---|---|
| DeepSeek-R1/V3 (671B total, 37B active) | Dual-Socket Xeon Gold 6454S (64 cores) | 64 | 4.51 | `llama.cpp` baseline reported in [31] |

Table 3.9: The only externally-sourced MoE data point we could independently verify; roughly 80× larger by active-parameter count than the MoE models staged for this study.

# Conclusion

Taken together, these results support one unifying claim: on CPU, prompt processing and text generation are governed by different physical constraints, and treating them as one workload leads to the wrong tuning advice. Prefill is compute-bound and well described by Amdahl-style saturation; generation is memory-bandwidth-bound, contention-sensitive enough to retrograde, and the specific technique – Flash Attention, KV-cache quantization, or weight-format choice – that helps on GPU or on disk can just as easily hurt once that bandwidth ceiling is the binding constraint. The Jetson comparison (Section 3.2) makes the mechanism vivid: model size, not thread count, decides which regime a workload is in. One result is explicitly incomplete rather than negative: the MoE bandwidth hypothesis (Section 3.8), which is motivated directly by the mechanisms measured here but not yet confirmed on our own hardware – a limitation this chapter shares with the excluded Raspberry Pi 3B attempt (Section 3.7), which failed for a hardware reason (insufficient RAM) rather than a measurement one. Chapter 4 turns these six empirical effects into a single calibrated equation and uses it to derive concrete deployment recommendations.

# Chapter 4

# A Pareto-Front Model and Practical Recommendations

## Introduction

Chapter 3 established six empirical effects one at a time: USL-shaped thread scaling, a compute/bandwidth regime split driven by model size, a Flash Attention regression at long context, a KV-cache quantization trade-off that flips sign with device and context length, a quantization-format anomaly independent of bit-width, and an untested but theoretically motivated MoE bandwidth advantage. This chapter's job is to turn those six separate observations into one calibrated equation, and then to turn that equation into concrete advice. Sections 4.1–4.5 build the model itself: a roofline formulation of generation throughput with USL-scaled compute on one side and a hard memory-bandwidth ceiling on the other, calibrated per device from a handful of measurements rather than fit universally across hardware we do not have access to. Section 5.1 then reads practical, device-specific recommendations directly off that model, closing the loop from measurement, to explanation, to actionable guidance.

## 4.1 What we can and cannot claim

The original goal for this line of work was a single equation that takes a CPU's specifications as input and outputs the optimal `llama.cpp` parameters. Rather than mapping specific CPU SKUs (e.g., "Ryzen 5600H") to parameters, which fails to generalize, we map *abstract hardware traits* – specifically physical core count, sustained memory bandwidth, and ISA vector width (AVX2/AVX-512/NEON) – to the model's calibration constants. What we present is a *functional form*, motivated directly by the effects measured in Section 3, whose free parameters are *calibrated per hardware class* from a handful of `llama-bench` runs. This turns a 100+ configuration grid search into roughly 6 calibration measurements plus arithmetic, and serves as the foundation for a broader hardware-to-parameters oracle as community benchmarks for new architectures are folded into the

Pareto front.

## 4.2 A roofline model with USL thread scaling

Let a device be characterized by the following calibration constants, each obtainable from a small sweep at $N \in \{1, 2, N_{\text{phys}}\}$ threads:

- $P_1(\text{model}, \text{quant})$ – single-thread prefill throughput (tok/s), and $\sigma_p, \kappa_p$ – USL coefficients fit to the prefill thread sweep (Section 3.1).
- $G_1(\text{model}, \text{quant})$ – single-thread generation throughput, and $\sigma_g, \kappa_g$ – USL coefficients fit to the generation thread sweep. Empirically $\kappa_g > \kappa_p$ always, reflecting generation's greater sensitivity to memory contention.
- $\text{BW}_{\text{eff}}$ – effective sustained memory bandwidth (GB/s), estimated as $\max_N \left[G(N)\right] \times B(\text{model}, \text{quant})$, i.e. backed out from the highest observed generation throughput rather than requiring a separate memory-bandwidth microbenchmark.

Prefill throughput is then modeled directly as USL-scaled single-thread throughput (Eq. 3.2):

$$T_{\text{pp}}(N) \;=\; P_1 \cdot \frac{N}{1 + \sigma_p(N-1) + \kappa_p N(N-1)} \tag{4.1}$$

Recall the intuition motivated in Chapter : a dense model must stream its entire quantized weight set from RAM for every single token it emits, while an MoE model only has to stream the weights of the experts its router actually selected. Generation throughput is modeled as a *roofline* that makes this concrete: the lesser of the same USL-scaled compute curve and a hard bandwidth ceiling, because whichever constraint binds first determines the observed rate:

$$T_{\text{tg}}(N) \;=\; \min\left[ G_1 \cdot \frac{N}{1 + \sigma_g(N-1) + \kappa_g N(N-1)} \;,\; \frac{\text{BW}_{\text{eff}}}{B(\text{model}, \text{quant})} \right] \tag{4.2}$$

where $B(\text{model}, \text{quant})$ is the number of bytes that must be streamed from RAM per generated token:

$$B_{\text{dense}} = N_{\text{params}} \times \frac{\text{bits/weight}}{8} \tag{4.3}$$

$$B_{\text{MoE}} = \left(N_{\text{active}} + N_{\text{shared}}\right) \times \frac{\text{bits/weight}}{8} \tag{4.4}$$

In equation terms, $B_{\text{dense}}$ is essentially the full quantized model size, while $B_{\text{MoE}}$ – covering only the router-selected experts plus the always-on shared layers – can be a small fraction of the model's total on-disk size. This is the formalized version of that argument: for two models of comparable output quality, one dense and one MoE, Eq. 4.2 predicts the MoE model's generation throughput ceiling is higher by roughly the ratio $N_{\text{params}}/N_{\text{active}}$ whenever generation is bandwidth-bound (which Section 3.2 shows is the common case on constrained CPUs). This structural advantage of MoE models is a direct

mathematical consequence of the roofline model when generation is bandwidth-bound, a regime we have empirically established as the default for constrained CPUs (Section 3.2). The MoE prediction itself is not yet locally validated; see the discussion in Section 3.8.

## 4.3 KV-cache as a second bandwidth term

For long-context generation, Eq. 4.2's bandwidth denominator gains a context-dependent term:

$$B(\text{model}, \text{quant}, \text{ctx}) \;=\; B(\text{model}, \text{quant}) \;+\; \underbrace{2 \cdot L \cdot H_{kv} \cdot d_{head} \cdot \frac{\text{bits}_{kv}}{8} \cdot \text{ctx}}_{B_{kv}(\text{ctx}, \text{quant}_{kv})} \tag{4.5}$$

with $L$ layers, $H_{kv}$ KV heads, and head dimension $d_{head}$. Quantizing the KV cache lowers $\text{bits}_{kv}$ and thus $B_{kv}$ – a pure win *if* generation is bandwidth-bound at that context length. But KV quantization also adds a per-access dequantization cost, which raises $G_1$'s effective inverse (i.e. lowers $G_1$) on the compute side of Eq. 4.2. Whether quantizing the KV cache helps or hurts is therefore exactly the sign of (bandwidth saved) − (compute added), which flips depending on device, context depth, and thread count. Section 3.4's laptop measurement – short context, 1 thread, compute-rich chip – sits on the compute-added side and shows a net loss; in fact, our data shows the net loss persists at all tested context depths on this hardware, suggesting the predicted flip may only occur on truly bandwidth-starved devices.

## 4.4 Worked example: calibrating the model for the Ryzen 5600H

To make the model concrete, we walk through the calibration for the Ryzen 5600H running Qwen2.5-7B-Instruct-Q4_K_M.

**Step 1: Measure single-thread baselines.** From our data: $P_1 = 7.81$ tok/s (pp512), $G_1 = 3.95$ tok/s (tg128).

**Step 2: Fit USL coefficients.** From the thread sweep in Section 3.1: $\sigma_p = 0.021$, $\kappa_p = 0.023$, $\sigma_g = 0.038$, $\kappa_g = 0.061$.

**Step 3: Estimate effective bandwidth.** The Q4_K_M format for a 7.6B-parameter model is ∼4.68 GB on disk. At the peak generation throughput of 8.83 tok/s (measured at 4 threads), the effective bandwidth is $\text{BW}_{\text{eff}} = 8.83 \times 4.68 \approx 41.3$ GB/s. This is ∼80% of the theoretical DDR4-3200 dual-channel peak of ∼51 GB/s, which is consistent with sustained (not burst) bandwidth.

**Step 4: Predict throughput at any thread count.** For $N = 6$ threads:

$$T_{\text{pp}}(6) = 7.81 \cdot \frac{6}{1 + 0.021(5) + 0.023(30)} = 7.81 \cdot \frac{6}{1.79} = 26.2 \text{ tok/s}$$

$$T_{\text{tg}}(6) = \min\left[3.95 \cdot \frac{6}{1 + 0.038(5) + 0.061(30)}, \frac{41.3}{4.68}\right]$$

$$= \min\left[3.95 \cdot \frac{6}{2.92}, 8.83\right] = \min[8.12, 8.83] = 8.12 \text{ tok/s}$$

The measured values were 25.15 tok/s (pp) and 6.90 tok/s (tg). The prefill prediction is within 4% of measured; the generation prediction is within 18%. The generation discrepancy is likely due to the USL fit underestimating the contention at 6 threads – a known limitation of fitting USL to only 5 data points.

**Step 5: Read the Pareto front.** Given the calibrated model, a user can now evaluate any (model, quant, threads, ctk, ctv) configuration and read off the predicted throughput, then compare against the quality and memory axes to find the Pareto-optimal configuration for their needs.

## 4.5 Defining the Pareto front

Given calibration constants for a device and a candidate list of (model, quantization, $N$ threads, ctk, ctv) configurations, each candidate maps to a point in a 3-dimensional objective space: (tokens/s from Eq. 4.2, perplexity delta from Section 3.6 or a completed local sweep, resident memory in GB). The Pareto front is the subset of candidates not *dominated* – i.e. for which no other candidate is simultaneously faster, more accurate, and smaller. The "optimal parameters for a given CPU" that motivated this whole project are then read directly off that front at whatever speed/quality/memory trade-off the user cares about.

# Conclusion

The model developed in this chapter is deliberately modest in its claims: it is a roofline-style equation with constants calibrated separately for each of three devices, not a single universal formula derived from first principles across all CPUs. What it buys, even at that modest scope, is real – it explains *why* the six effects of Chapter 3 occur in terms of two competing constraints (USL-scaled compute and a hard bandwidth ceiling) rather than leaving them as six unrelated observations, it correctly predicts measured Ryzen throughput within 4–18% from just five calibration points (Section 4.5), and it converts directly into the seven concrete, device-aware recommendations above. Its most consequential prediction – that MoE models should be structurally favored on bandwidth-starved CPUs – is also its least tested: Chapter 5 takes up that gap directly, alongside the model's other honest limitations and the concrete next steps for closing them.

# Chapter 5

# Discussion

## Introduction

Chapters 3 and 4 presented what the data shows and the model built to explain it. This chapter steps back and asks what that combination actually supports, and what it does not. Section 5.2 discusses the findings honestly against their scope – an opportunistic, three-device fleet, a single inference engine, and several benchmark sweeps that did not finish – and argues that the roofline-USL model's value lies less in the specific numbers it reproduces than in the mechanism it makes explicit: that CPU inference splits cleanly into a compute-bound and a bandwidth-bound regime, and that knowing which regime a workload is in predicts, rather than merely describes, whether a given optimization helps or hurts. Section 5.3.1 then lays out the concrete work needed to close the gaps this discussion identifies, ordered from the near-term experiments this study ran out of time for to the longer-term directions that would follow if they succeed.

## 5.1 Lessons Learned

Based on the empirical data and the roofline model above, we distill the following concrete recommendations for users running `llama.cpp` on consumer hardware:

1. **Disable Flash Attention on CPU at long context.** Our data in Section 3.3 shows Flash Attention is a 3× slowdown for generation at 15K-token context on the Ryzen 5600H. Use `-fa 0` (the default in many frontends is `-fa 1`, which is harmful on CPU).

2. **Do not quantize the KV cache on compute-rich CPUs.** Our data in Section 3.4 shows KV quantization is a net loss at all tested context depths on the Ryzen 5600H. Use `-ctk f16 -ctv f16` unless you are on a truly bandwidth-starved device.

3. **Use physical core count, not logical core count, for thread count.** Our USL fits in Section 3.1 show throughput saturates at the physical core count; adding

hyperthreads buys nothing for prefill and hurts generation. Use `-t` equal to the number of physical cores.

4. **On bandwidth-starved CPUs, single-threaded generation may be optimal.** The Pentium G5420 data in Section 3.1 shows monotonic retrograde scaling for generation – adding threads makes it slower. On such hardware, use `-t 1` for generation.

5. **On legacy CPUs without AVX-512/VNNI, legacy quantization formats may be faster.** The Pentium data in Section 3.5 shows `Q4_0` is 3.6× faster than `Q4_K_M` at 1 thread. If you are on an older CPU and speed is the priority, prefer legacy formats; if quality is the priority, prefer K-quants.

6. **Prefer MoE models on bandwidth-starved hardware – provisionally.** The roofline model above predicts MoE models should be disproportionately advantaged on bandwidth-starved CPUs because they stream fewer bytes per token. This is theoretically motivated and loosely consistent with one external CPU-MoE benchmark on a much larger model (Section 3.8), but it has not been locally validated on the specific models this study staged. Treat it as a strong hypothesis to test on your own hardware, not a settled recommendation.

7. **Match model size to the edge device's memory budget.** This follows directly from the roofline model's OOM/thrashing behavior above: on memory-starved hardware, a model that does not fit in RAM alongside its KV cache will thrash or fail outright regardless of any other tuning. Published third-party benchmarks suggest an 8 GB Pi 5 can sustain low-single-digit avg. tok/s on 7B-class Q4 models [29, 30] – treat that as a rough external data point, not a number we verified.

## 5.2 Limitations

The central result of this thesis is not any single number in Chapter 3, but the mechanism the roofline-USL model in Chapter 4 makes explicit: CPU inference is not one workload but two, governed by different physical constraints, and a technique's effect on throughput depends on which constraint is binding. That framing does real explanatory work. It is what lets a single equation account for four superficially unrelated observations – retrograde thread scaling, a Flash Attention regression, a KV-cache quantization trade-off that flips sign, and a quantization-format anomaly independent of bit-width – as four faces of the same compute/bandwidth boundary, rather than as four disconnected curiosities. It is also what turns those observations into actionable, device-conditional advice (Section 5.1) instead of a single blanket recommendation that would be wrong on at least some of the hardware this thesis tested. At the same time, the model's scope is narrower than its ambition. Three limitations matter most. First, the model is calibrated, not derived: $\sigma$, $\kappa$, and $\mathrm{BW}_{\mathrm{eff}}$ are fit separately to each of three devices from a handful of local measurements, and nothing in this thesis regresses those constants against raw hardware specifications (core count, cache size, memory channels, ISA width) in a way that would let the model predict a fourth device's behavior *without* measuring it first. The "oracle that takes a CPU spec and outputs optimal flags," which motivated this project at the outset (Chapter ), is therefore not yet what this thesis delivers –

what it delivers is a much cheaper way to calibrate that oracle per device (roughly six measurements instead of a full grid search), not a way to skip calibration entirely. Second, the thesis's most consequential prediction – that Mixture-of-Experts models should be structurally advantaged on exactly the bandwidth-starved hardware this study focused on – is a direct, well-motivated consequence of the bandwidth mechanics measured in Section 3.2, but it was never confirmed on the study's own hardware (Section 3.8), and the one external data point available (a 671B-parameter model on server-class dual-socket Xeon hardware) is too far removed in both model scale and hardware class to validate the effect's magnitude for the consumer and embedded devices this thesis is actually about. Third, every finding here was measured on one inference engine (`llama.cpp`); Section 3.5 explicitly cannot distinguish whether the quantization-format anomaly it reports is a property of the hardware and the underlying arithmetic, or an artifact of how this one engine dispatches its CPU kernels. These are not reasons to discount the findings – every effect reported in Chapter 3 is a real, reproducible measurement, and the model correctly predicts held-out Ryzen throughput within 4–18% from five calibration points (Section 4.5) – but they do bound what can be claimed from them. The honest scope of the underlying data matters enough to state explicitly rather than leave implicit: Table 5.1 gives the complete accounting of what was run to completion on each machine, and every quantitative claim in this thesis that is not explicitly attributed to a third-party source is drawn only from the rows marked complete there. Concretely, this coverage

| Machine | Completed | Not run / incomplete |
|---|---|---|
| Laptop (Ryzen 5600H) | KV-cache and attention sweeps, f16 KV, fa=0/1, all 3 depths, full 5-thread sweep. | Quantized-KV: 3/9 pairings, thread=1 only. Perplexity, model-comparison sweeps: not run. |
| Desktop (Pentium G5420) | `Q4_0` quant sweep (full, 5 threads). `Q3_K_M` (11/12 rows). | `Q4_K_M`: repeatedly restarted, never reached thread > 4. Other sweeps: not run. |
| Jetson Nano | Model-size comparison sweep, both target models, 3-thread sweep (full). | KV-cache and attention sweeps: 0 rows, file present but empty. |

Table 5.1: Data coverage by machine and sweep; every unattributed claim in this thesis is drawn only from rows marked complete.

implies the following specific boundaries on the current model:

- **Hardware Fleet Scope.** Three devices produced usable data (Ryzen 5600H, Pentium G5420, Jetson Nano). This is not enough to fit a hardware-spec-to-parameters mapping with any statistical confidence – the per-device calibration constants in Section 4.1 are exactly that, per-device, not yet a general function of core count / bandwidth / ISA. Expanding to a broader, more standardized fleet (including a working Raspberry Pi 4/5 run, a modern ARM board with SVE/SME, and an AVX-512 x86 chip) is the top priority for turning this into a genuine zero-shot predictor.
- **Serving and Concurrency.** Every measurement in this thesis is single-user, batch-size-1 generation. The system-level and serving sweeps – which targeted CPU pinning,

memory-mapping, multi-process contention, and speculative decoding – did not produce usable data on any machine (Table 5.1), so the model says nothing yet about concurrent-request serving.

- **Quality Axis Calibration.** The perplexity deltas used to define the quality axis of the Pareto front are currently calibrated using established literature values for the GGUF ecosystem [34]. Completing local perplexity sweeps across all tested quantization formats will further ground this axis in hardware-specific measurements.
- **Contention and Scheduling.** The USL coefficients in Section 3.1 reflect default OS scheduling. Exploring pinned/isolated configurations and NUMA interleaving on multi-socket or chiplet-based systems would refine the coherency penalty ($\kappa$) for production deployment scenarios.
- **Flash Attention on Other Hardware.** Our Flash Attention findings in Section 3.3 are specific to the Ryzen 5600H. It is possible that on hardware with smaller L3 caches (e.g., the Pentium or Jetson), Flash Attention's benefit could outweigh its overhead. This is an open question for future work.
- **KV-Cache Quantization on Bandwidth-Starved Hardware.** Our KV-cache quantization findings in Section 3.4 are specific to the compute-rich Ryzen 5600H. The roofline model predicts a sign-flip on bandwidth-starved hardware, but we have not yet tested this. This is a high-priority next experiment.

## 5.3 Perspectives

The limitations above split naturally into two kinds: things this study already knows how to do and simply ran out of time for, and things that would require a materially larger or longer-running effort. This section addresses both, ordered from nearest-term to most ambitious.

### 5.3.1 Near-Term: Completing the Current Study

Building directly on the roofline model and Pareto framework, the following concrete next steps would close the specific gaps identified in Section 5.2:

1. **Expanding the hardware fleet** to include standardized edge and server platforms, enabling the regression of USL parameters directly against physical hardware specifications for zero-shot parameter prediction. Priority targets: a working Raspberry Pi 4/5 run, an x86 CPU with AVX-512 VNNI, an Apple Silicon Mac, and a multi-socket server with NUMA.
2. **Executing comprehensive local empirical benchmarks of diverse MoE architectures** to directly validate and refine the bytes-touched-per-token model, particularly focusing on router overhead and shared-layer bottlenecks. The staged models

(Qwen1.5-MoE-A2.7B, DeepSeek-V2-Lite, DeepSeek-Coder-V2-Lite) will be benchmarked against dense equivalents on the same hardware.

3. **Completing local perplexity sweeps** to replace literature-based quality calibrations with hardware-specific measurements using `llama-perplexity` on the WikiText-2 dataset.

4. **Investigating the impact of advanced CPU scheduling**, CPU pinning, and NUMA interleaving on the coherency penalties observed during memory-bandwidth-bound generation.

5. **Testing Flash Attention and KV-cache quantization on bandwidth-starved hardware** (Pentium, Jetson) to validate the roofline model's predicted sign-flips.

### 5.3.2 Longer-Term Directions

The near-term items above are things this study already knows how to do. This subsection takes a longer view: assuming they are completed and the roofline-USL model holds up, where could this line of work go next? The following seven directions are ordered roughly from nearest-term extensions of the existing framework to more ambitious departures from it.

#### From per-device calibration to a true zero-shot predictor

The central honest limitation of this thesis (Section 5.2) is that $\sigma$, $\kappa$, and $\mathrm{BW}_{\mathrm{eff}}$ are calibrated separately for each device from a handful of local `llama-bench` runs, not derived from a device's raw specifications. Turning this into a genuine zero-shot predictor – one that takes "6 cores, 2-channel DDR5, AVX-512" as input and outputs calibration constants without ever touching that hardware – requires calibrating enough devices that a regression against raw specs (core count, cache sizes, memory channels, ISA extensions) becomes statistically meaningful. Three partially-sampled devices, as this thesis has, is nowhere near enough; a follow-on study with on the order of twenty to thirty calibrated devices, spanning several CPU vendors and generations, is the natural next scale for this work.

#### A living, community-contributed calibration dataset

Reaching that scale through one researcher's personal hardware fleet is impractical, which is precisely the gap that led earlier drafts of this thesis to lean on external benchmark numbers that turned out to be difficult or impossible to verify (Section 3.8). A more sustainable path is to package the resumable calibration harness described in Section 2.6 as a lightweight, opt-in "auto-tune" companion to `llama.cpp` – something a user could run once after installation to get personalized flag recommendations for their own machine, with the option to contribute the resulting (anonymized) calibration constants back to a shared, versioned public dataset. This would directly address the verifiability problem

this thesis ran into: every data point would come with a known provenance, a known harness version, and a known schema, rather than being a number copied from a blog post or forum thread.

### From single-user batch-1 to concurrent serving

Every measurement in this thesis assumes a single user generating one response at a time (batch size 1), which is the right assumption for a personal chat application but not for a small on-premises server handling several users at once. The system-level sweeps of the instrumentation (Chapter 2) were designed to probe exactly this – CPU pinning, NUMA interleaving, and multi-process contention – but none produced usable data in this round (Table 5.1). Completing them is listed as near-term future work above; the longer-term direction is extending Equation 4.2's roofline itself to a concurrent-request setting, where the bandwidth ceiling is shared across simultaneous generations rather than claimed by one, and the USL coherency term $\kappa$ plausibly needs a second contention source (inter-process, not just inter-thread) added to it.

### Generalizing the bytes-per-token lens beyond MoE

The MoE bandwidth argument in Chapter 4 rests on a single mechanism: a technique that reduces how many bytes must move through memory per generated token should, on memory-bound CPU hardware, produce a proportional throughput gain. Mixture-of-Experts routing is one way to reduce that byte count, but not the only one. Structured pruning, layer-skipping or early-exit techniques, and the small draft model used in speculative decoding all reduce effective bytes-touched-per-token through different mechanisms. A unified "effective active parameters" framework that treats MoE routing, pruning, and speculative decoding as three instances of the same underlying lever would let the roofline model predict the CPU speedup from any of them without re-deriving the argument from scratch each time.

### Modeling conversations, not snapshots

Every context-depth result in this thesis (Sections 3.3 and 3.4) is a snapshot: throughput measured at one fixed context length. A real chat session is not a snapshot – it is a trajectory, with the KV cache growing turn by turn over what might be a fifty-turn conversation. Because both the Flash Attention regression and the KV-cache quantization sign-flip are context-length-dependent, a configuration that is optimal at turn one of a conversation is not guaranteed to still be optimal at turn thirty. Extending the Pareto front (Section 4.5) from a single-point prediction to a trajectory – predicted throughput as a function of turns elapsed, for a given configuration – would let a practitioner choose settings based on the conversation lengths they actually expect, rather than a single benchmark point that may not represent real usage.

### Joint performance-energy rooflines for edge deployment

This thesis measured throughput exclusively; it did not measure power draw or thermal throttling, even though the Jetson Nano – one of its three target devices, and the class of low-power embedded board that the excluded Raspberry Pi 3B also belongs to – is exactly the class of hardware where power budget and sustained thermal output often matter as much as raw speed – a battery-powered or fanless deployment can be throughput-optimal on paper and still be unusable in practice if it draws too much power or throttles after ninety seconds. A natural extension of the roofline model already in this thesis is a second, joint roofline in performance-per-watt rather than raw performance, which would very likely reorder some of the recommendations in Section 5.1 – the fastest configuration and the most power-efficient one are not always the same configuration.

### Testing how much of this is `llama.cpp`-specific

Every finding in this thesis – the Flash Attention regression, the `Q4_0`-versus-K-quant kernel effect, the USL coefficients themselves – was measured on one inference engine. Section 3.5 explicitly flags that we do not know whether the quantization-format anomaly reflects a property of the hardware and the algorithm, or an artifact of how `llama.cpp` specifically dispatches its CPU kernels. Repeating a subset of this study's measurements on at least one other CPU inference engine (for example, ONNX Runtime or MLC-LLM) would separate genuine hardware/algorithm physics from implementation-specific quirks, and would tell us how much of the roofline-USL framework developed here is a statement about CPU inference in general versus a statement about this one, widely-used but not unique, engine.

## Conclusion

This chapter has tried to state plainly what Chapters 3 and 4 do and do not support: a real, reproducible, mechanistically-explained account of thread scaling and memory-bandwidth effects on three specific CPUs, wrapped around a genuinely untested central hypothesis about MoE models and calibrated on a fleet too small to generalize without further work. Neither half of that summary should be read as undermining the other – the measured effects stand on their own regardless of whether the MoE prediction holds up, and the model's honesty about its own limits is what makes the near-term work above tractable rather than open-ended. The final chapter closes the thesis by drawing these threads together.

# Chapter 6

# Conclusion

Three consumer and embedded CPUs benchmarked with a resumable `llama.cpp` harness were enough to recover four concrete and reproducible effects governing CPU-only LLM inference: (1) compute-bound prefill that saturates near the physical core count and follows Gunther's Universal Scalability Law; (2) memory-bound generation that can retrograde as threads are added, most dramatically on bandwidth-starved processors; (3) quantization choices (both weight and KV-cache format) whose speed impact depends on which side of the compute/bandwidth boundary a given device and context length fall on; and (4) Flash Attention, which is a performance regression on CPU at long context. We formalized these effects into a roofline-style throughput model with per-device calibration constants, rather than a single universal equation, because the data honestly available does not yet support more than that. The model's most interesting prediction – that Mixture-of-Experts architectures should be structurally advantaged on exactly the bandwidth-starved hardware this study focused on – remains directly untested on our own hardware and is loosely, not strongly, corroborated by outside sources; validating it is the clearest next step for turning this from a benchmarking exercise into the MoE-on-CPU research program it was originally motivated by.

Looking forward, this work opens several concrete avenues for future research. In the near term, the most immediate priority is expanding the hardware fleet – specifically to include a working Raspberry Pi 4/5, an AVX-512-equipped x86 chip, and Apple Silicon – to complete the interrupted benchmark sweeps, run local perplexity evaluations, and directly test the roofline model's predicted sign-flips for Flash Attention and KV-cache quantization on genuinely bandwidth-starved devices. Concurrently, executing comprehensive local benchmarks of the staged MoE models against their dense equivalents is essential to empirically validate the bytes-touched-per-token hypothesis.

In the longer term, the ultimate goal is to evolve this framework from a per-device calibration tool into a true zero-shot predictor. This will require regressing the USL and roofline constants against raw hardware specifications across a community-contributed dataset of twenty to thirty diverse architectures. Beyond single-user batch-1 generation, extending the roofline model to account for concurrent serving, multi-process contention, and joint performance-per-watt constraints will be critical for real-world edge deployments. Finally, generalizing the "effective active parameters" lens to encompass tech-

niques like speculative decoding and early-exit mechanisms, and testing these findings across alternative inference engines, will determine how much of this framework reflects fundamental CPU physics versus implementation-specific quirks of `llama.cpp`. Ultimately, this thesis provides the mechanistic foundation and the tooling to turn CPU-only LLM deployment from an exercise in trial-and-error into a predictable, hardware-adaptive science.

# Bibliography


[1] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Guoliang He, Xupeng Miao, Ana Klimovic, Bin Cui, Binhang Yuan, and Eiko Yoneki. Demystifying cost-efficiency in LLM serving over heterogeneous GPUs. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 27534–27552. PMLR, 13–19 Jul 2025.

[2] Muhammad Adnan, Rohan Mahapatra, Prashant J. Nair, Daniel Berger, Pantea Zardoshti, Rodrigo Fonseca, and Esha Choukse. Cascade: Exploiting slo-aware latency budget for fair and high goodput llm inference serving, 2026.

[3] Boxiao Du, Boning Huangfu, Yizhou Luo, Chen Chen, Zijun Li, Minchen Yu, Xiaoyi Fan, and Minyi Guo. Goodserve: Towards high-goodput serving of agentic llm inferences over heterogeneous resources, 2026.

[4] NVIDIA. Overview — TensorRT-LLM performance. https://nvidia.github.io/TensorRT-LLM/latest/developer-guide/perf-overview.html, 2026. Accessed: 2026-08-20.

[5] Mariam Rakka, Marios Fournarakis, Olga Krestinskaya, Jinane Bazzi, Khaled N Salama, Fadi Kurdahi, Ahmed M Eltawil, and Mohammed E Fouda. Mixed-precision quantization for language models: Techniques and prospects. *arXiv preprint arXiv:2510.16805*, 2025.

[6] Pierre V. Dantas, Lucas C. Cordeiro, and Waldir S. S. Junior. A review of state-of-the-art techniques for large language model compression. *Complex & Intelligent Systems*, 11(9):407, August 2025.

[7] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. Understanding performance implications of llm inference on cpus. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*, pages 169–180, 2024.

[8] Zhen Bi, Xueshu Chen, Luoyang Sun, Yuhang Yao, Qing Shen, Jungang Lou, and Cheng Deng. RooflineBench: A benchmarking framework for on-device LLMs via roofline analysis. *arXiv preprint arXiv:2602.11506*, 2026.

[9] Feiyang Chen and Haibo Chen. SMEPilot: Characterizing and optimizing LLM inference with scalable matrix extensions. *arXiv preprint arXiv:2606.16332*, 2026.

[10] Xiang Zhang, Wei Li, Yu Wang, Hao Chen, Xuan Liu, Yao Zhao, and Wenxi Sun. HeteroMosaic: Exposing and exploiting heterogeneous execution opportunities for energy-efficient edge LLM inference. *arXiv preprint arXiv:2607.12839*, 2026.

[11] Uygar Kurt. Which quantization should I use? a unified evaluation of llama.cpp quantization on Llama-3.1-8B-Instruct. *arXiv preprint arXiv:2601.14277*, 2026.

[12] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Aaron Jezghani, Jeffrey Young, Christopher J. Hughes, Tushar Krishna, and Hyesoon Kim. Flexinfer: Flexible llm inference with cpu computations. In M. Zaharia, G. Joshi, and Y. Lin, editors, *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025.

[13] Georgi Gerganov. ggerganov/llama.cpp: Port of facebook's LLaMA model in C/C++, 2023. GitHub repository.

[14] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, 1967.

[15] Yassine Hattay. Hardware-adaptive pareto fronts for CPU-only LLM inference: Data and code. https://github.com/yassine-hattay/LLMs_inference_on_CPU, 2026. Accessed: 2026-08-22.

[16] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[17] MLC team. MLC-LLM, 2023-2025.

[18] Marcin Chrapek, Marcin Copik, Etienne Mettaz, and Torsten Hoefler. Confidential llm inference: Performance and cost across cpu and gpu tees. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*, pages 84–98, 2025.

[19] RunAIHome. Q4 vs Q5 vs Q6 vs Q8 quantization: Real quality loss numbers for local LLMs. https://runaihome.com/blog/quantization-q4-q5-q6-q8-quality-loss-2026/, 2026.

[20] Qwen Team. Qwen2.5 technical report, 2024.

[21] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 2009.

[22] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, 2007.

[23] Elias Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

[24] Ji Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *MLSys*, 2024.

[25] Mistral AI. Mixtral of experts, 2024.

[26] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.

[27] Qwen Team. Qwen1.5-MoE: A strong, economical, and efficient mixture-of-experts language model, 2024.

[28] Pablo Prieto and Pablo Abad. Edge deployment of small language models, a comprehensive comparison of cpu, gpu and npu backends. *arXiv preprint arXiv:2511.22334*, 2025.

[29] Jeff Geerling. LLMs accelerated with eGPU on a Raspberry Pi 5 / AI benchmarks. https://github.com/geerlingguy/ai-benchmarks, 2024.

[30] LocalAIMaster. Raspberry Pi 5 LLM benchmarks: 12 models, real tokens. https://localaimaster.com/blog/llm-raspberry-pi-5, 2026.

[31] GitHub / LocalLLaMA Community. KTransformers: DeepSeek-V3/R1 CPU inference benchmarks, 2024.

[32] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

[33] Tri Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[34] Thomas Baruzier. Qwen2.5-7B-Instruct-GGUF: Perplexity and quantization metrics. https://huggingface.co/ThomasBaruzier/Qwen2.5-7B-Instruct-GGUF, 2024.